



MASTER'S DEGREE THESIS

Dipartimento di Informatica

Corso di Laurea Magistrale in Informatica

Modular Deep Reservoir Computing

Author:

Vincenzo Gargano

Supervisors:

Prof. Claudio Gallicchio

Dott. Andrea Ceni

Dott. Andrea Cossu,

Academic Year 2024/2025

Abstract

...

Contents

Chapter 1

Introduction

1.1 Motivation and Objectives

Nowadays, Artificial Intelligence (AI) became the main driver of innovation in many fields, at the point that the growth of research became exponential in the last decade. This fully aligns with the current state of our world, in fact, we are observing a paradigm shift in the way we interact with technology (or world), through the use of AI.

Within AI, whose main goal is not only to understand *intelligence*, but to embed it into agents capable of performing complex tasks through *rational reasoning*, *planning* and *acting*, the subfield of Machine Learning (ML) has emerged as the dominant approach to mimicking biological intelligent behaviour. Neural Networks (NN) *learn* complex patterns from large amounts of real-world data, instead of relying on explicit rule based approaches, in order to perform *inference* and return prediction on unseen data.

The last years have seen a surge in popularity of generative models, after a first wave of discriminative models that made popular the field of Deep Learning (DL), and introduced GPU computation in 2009 [1]. Particularly, deep generative models have been applied with unbelievable results in many fields, from image generation with *Diffusion Models* [2] to text generation with *Transformers* [3] and protein folding with *AlphaFold* [4] and many others domains, achieving state of the art results. However, deep learning models are known to be expensive in both training and inference, requiring large amount of computational resources (GPU production) and energy consumption [5].

Forse dovrei concentrarmi più su RNN dall'inizio?

1.2 Reservoir Computing for Efficient AI

Sequential data processing remains a fundamental challenge in machine learning, from natural language to time series forecasting. While Recurrent Neural Networks (RNNs) have been the traditional approach for modeling temporal dependencies, training them to capture long-term patterns has proven notoriously difficult due to vanishing and exploding gradients. Although architectures like LSTMs and GRUs

mancano citazioni e è ripetitivo rispetto al background

have partially addressed these issues through gating mechanisms, and Transformers have recently dominated the field with their attention-based approach, these solutions come at the cost of increased architectural complexity and computational demands.

An alternative paradigm that has gained attention is Reservoir Computing (RC)[1], which offers a fundamentally different approach to sequential processing. By maintaining a fixed, randomly initialized recurrent network as a "reservoir" of dynamics, and training only a simple linear readout layer, RC circumvents the gradient-based training of recurrent connections entirely. This framework not only simplifies the learning process but also draws inspiration from neuroscience and physical computing systems, where rich dynamics emerge naturally without explicit optimization.

Despite its computational efficiency and theoretical elegance, vanilla RC models face their own set of limitations, particularly in scaling to complex tasks that require multiple levels of temporal abstraction[2]. This motivates the exploration of *deep* variants [3] of reservoir computing architectures, which aim to combine the efficiency of the RC paradigm with the representational power of hierarchical processing. This thesis investigates such deep reservoir architectures, examining how stacking multiple reservoir layers can enable learning at different timescales while maintaining the training efficiency that makes RC attractive.

After finishing experimental part...

1.3 Scientific contribution

1.4 Structure of the work

elenco puntato con quello che si dirà in ogni capitolo della tesi

Chapter 2

Background

In this chapter, we will review the main concepts and theory needed to understand the rest of this thesis. We will start with a brief introduction to the Dynamical Systems (DS) theory and an overview of Machine Learning concepts and foundational models that will be useful to understand the rest of the work.

2.1 The Inherent Dynamism of Natural Systems

Dynamics are an intrinsic aspect of reality, and the universe can be thought of as a complex, non-linear dynamical system where the state of any part is constantly changing over time due to interactions with other parts. Historically, the mathematical modelling of such systems began with the study of simple differential equations as a method to solve the two-body problems, but many natural phenomena are essentially Complex Systems (CS), emerging from simple interactions between single components of the environment, which exhibits complex dynamics that evolve through time: such as motion of planets, flow of fluids, weather changes, growth of population [6] and many other phenomena.

Since dynamics is such an important aspect of natural systems, many real world processes can be modelled as dynamical systems, eg. the design of a control system, the stock market [7] and also the human brain [8] can be modeled as a dynamical system.

Troughout the last century the analysis of sequential data, commonly referred as *time series*, became increasingly critical for understanding dynamical behaviours. Consequently specific methodological frameworks were developed to model such data. The approaches encompass techniques such as Autoregressive (AR) models which employ linear regression on data sequences, extending to more sophisticated frameworks including State-Space Models [9], originally developed for Control Theory applications, and Hidden Markov Models (HMM) [10]. The latter served as foundational precursors to Recurrent Neural Networks [11] (RNNs). The aforementioned models are the basis of modern neural-based sequential modeling.

2.2 Dynamical Systems review

A dynamical system is a system which evolves over time we can describe its changes through **state variables**, usually many models are constructed such that only the fundamental state variables are considered, for example *biological spiking neurons* modeled as in [12] are described by a simple 2-D dimensional ODE, where the two state variables are the membrane potential and a recovery variable. ignoring existing other possible variables, such gradient of ion concentration, temperature, for the sake of obtaining a large scale simulation.

There are two types of dynamical systems: *Differential Equations*, which are continuous in time, and *Difference Equations* or *Iterated Maps*, which are discrete in time. The most used in a wide range of scientific fields are the former, which are further distinguished in Ordinary Differential Equations (ODE) and Partial Differential Equations (PDE), the difference is that in PDE we have two or more independent variables.

This work will treat ODEs in their discretized form which we'll see in section ?? to obtain its approximated solution.

Formally, we define an ODE as:

$$\frac{d\mathbf{x}}{dt} = f(\mathbf{x}, t) \quad (2.1)$$

where $\mathbf{x} \in \mathbb{R}^n$ is the state vector, t is the time and $f : \mathbb{R}^n \times \mathbb{R} \rightarrow \mathbb{R}^n$ is a vector field grouping all the state variables that describe the evolution of the system.

questo va tolto, forse
meglio non andare
oppo in dettaglio
in queste cose

An example of ODE is the well-known Damped Harmonic Oscillator:

$$\frac{d^2x}{dt^2} + 2\beta \frac{dx}{dt} + \omega_0^2 x = 0 \quad (2.2)$$

anche questo va
tolto?

where x is the position of the oscillator, β is the damping coefficient and ω_0 is the natural frequency of the oscillator.

A Differential Equation is *autonomous* if the function f does not depend explicitly on the independent variable, such as time; otherwise it is *non-autonomous*. We can switch from a non-autonomous to an autonomous system by adding the independent variable as a new variable to the state vector, for example, the non-autonomous system of equation ?? can be rewritten with a change of variable.

Generally, a differential equation has an infinite number of solutions, since it depends on the state at a given time, if we define a certain state at time t_0 as **initial condition** such that $\mathbf{x}(t_0) = \mathbf{x}_0$. Then the solution of the ODE is called *initial value*

problem.

Definition 1. An initial value problem (IVP) is defined as the pair $(\frac{dx}{dt} = f(\mathbf{x}, t), \mathbf{x}(t_0) = \mathbf{x}_0)$, where $\frac{dx}{dt} = f(\mathbf{x}, t)$ is a differential equation and $\mathbf{x}(t_0) = \mathbf{x}_0$ is the initial condition at time t_0 .

In order to solve the IVP, we need to find a function $\mathbf{x}(t)$ that satisfies both the differential equation and the initial condition.

2.2.1 Stability

Stability [13] is a fundamental concept in the study of dynamical systems; it describes how a system responds to small perturbations or changes in initial conditions.

A system is said to be **stable** if small perturbations in the initial conditions lead to small changes in the system's behaviour over time. On the other hand, a system is **unstable** if small perturbations can lead to large deviations in behaviour.

Considering the generic equation ??, an equilibrium point $\mathbf{x}^*$ is a point where $f(\mathbf{x}^*) = 0$. The stability of this equilibrium point can be analysed using the following definitions:

- **Lyapunov Stability:** An equilibrium point $\mathbf{x}^*$ is Lyapunov stable if, for every $\epsilon > 0$, there exists a $\delta > 0$ such that if $\|\mathbf{x}(t_0) - \mathbf{x}^*\| < \delta$, then $\|\mathbf{x}(t) - \mathbf{x}^*\| < \epsilon$ for all $t \geq t_0$. The system remains close to the equilibrium point remains close to the equilibrium point for all future times within the ϵ bound.
- **Asymptotic Stability:** An equilibrium point $\mathbf{x}^*$ is asymptotically stable if it is Lyapunov stable and, in addition, there exists a $\delta' > 0$ such that if $\|\mathbf{x}(t_0) - \mathbf{x}^*\| < \delta'$, then $\lim_{t \rightarrow \infty} \mathbf{x}(t) = \mathbf{x}^*$. The system remains close to the equilibrium point, but it also converges to it over time.
- **Uniform Stability:** An equilibrium point $\mathbf{x}^*$ is uniformly stable if the δ in the Lyapunov stability definition can be chosen independently of the initial time t_0 .
- **Instability:** An equilibrium point $\mathbf{x}^*$ is unstable if it is not Lyapunov stable.

To analyse the stability of an equilibrium point, we can use linearization, which involves approximating the system near the equilibrium point using its Jacobian matrix. The Jacobian matrix $\mathbf{J}$ is defined as:

$$\mathbf{J} = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\mathbf{x}=\mathbf{x}^*} \quad (2.3)$$

The eigenvalues of the Jacobian matrix provide insights into the local behaviour of the system near the equilibrium point.

aggiungere immagine
per far capire meglio

forse è meglio usare
un'immagine invece
di scrivere tutto
questo formalismo

dovrei descrivere
come le parti immaginarie
e reali degli
autovalori cambiano
il comportamento del
sistema?

2.2.2 Modeling

Modelling is a central methodology in the study of complex systems, translating real-world observations and hypotheses into a formalised mathematical model, even though simplified, can be crucial for studying emergent behaviours, particularly in natural systems where we cannot change the initial state, or the environmental variables that influence it.

The Hodgkin-Huxley model [14] describes the dynamics of the neuronal membrane potential through an equivalent electrical circuit. The membrane can be represented as a capacitor C_m in parallel with several ion channels, each modelled as a voltage-dependent conductance in series with a battery representing the ion's reversal potential.

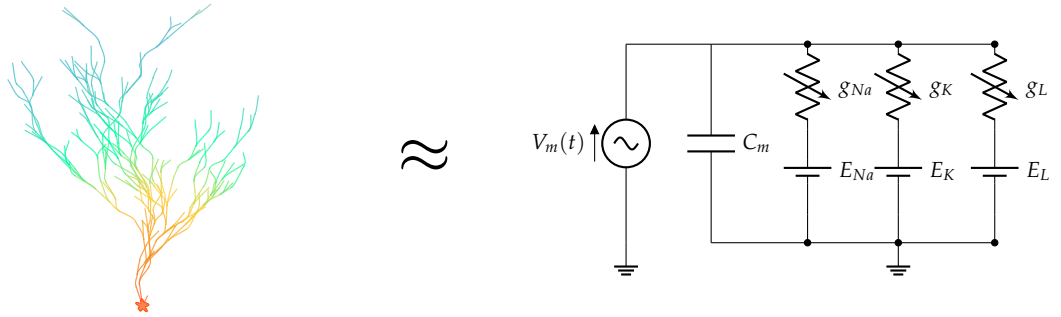


FIGURE 2.1: Biological neuron and its Hodgkin-Huxley equivalent circuit representation.

The corresponding differential equation describing the membrane potential dynamics is:

$$C_m \frac{dV_m}{dt} = I_{ext} - g_{Na}(V_m - E_{Na}) - g_K(V_m - E_K) - g_L(V_m - E_L) \quad (2.4)$$

where I_{ext} is the external input current, and each conductance g_i is voltage and time dependent, governed by additional gating variables.

However, as previously mentioned, natural systems are often complex; as in this case, we have a four-dimensional non-linear coupled system. Accurate modelling can be computationally infeasible, thus often simplified models are preferred for large-scale simulations, such as the Izhikevich neuron model [12] that captures essential spiking dynamics with only two state variables.

quanto in dettaglio?

2.2.3 Discretization Methods

Finding an analytical solution for an ODE is unfeasible in many cases; thus, we need to rely on numerical methods to approximate the solution. Discretising the time variables in small steps of size τ , there are many ways to approximate the solution of an ODE, but two main classes exist: **Explicit** and **Implicit** methods.

Implicit methods are more stable and accurate for numerically unstable equations (also known as stiff systems¹), but require solving a system of equations at each timestep, which can be computationally expensive. On the other hand, Explicit methods compute the state at the timestep $t + 1$ directly from the current state at time t , for example the well known **Euler's method** [15] approximates the solution as:

$$\mathbf{x}(t + \tau) = \mathbf{x}(t) + \tau f(\mathbf{x}(t), t) \quad (2.5)$$

The main methods used for this work are the Explicit Euler's method, which introduces some error, the local discretisation error, is $\|\mathbf{x}(t + \tau) - \mathbf{x}_{\text{approx}}(t + \tau)\| = O(\tau^2)$, while the global error over N steps is $O(\tau)$. This method is preferred since we're trying to lowering computation cost as much as possible, however, in order to keep the computation stable, we need to use small time steps τ . Of course exists a wide range of more advanced methods, such as the **Runge-Kutta** methods or even Adaptive Runge-Kutta [15] which provide higher order accuracy if needed.

¹Stiff systems are those where some terms change very rapidly compared to others.

2.3 Overview of Machine Learning

Machine Learning (ML) is a subfield of Artificial Intelligence (AI) that focuses on statistical methods and algorithms capable of *learn* from past experiences. First, we need a rigorous definition of what it means to learn; Mitchell [16] defines it as: "A computer program is said to **learn** from experience **E** with respect to some class of tasks **T** and performance measure **P**, if its performance at tasks in **T**, as measured by **P**, improves with experience **E**."

ML can tackle a much wider range of tasks that are difficult to solve with explicit rules-based programming; in a certain sense, it bypasses some level of abstraction, we're not learning the task T directly, but we're sampling from the E space, **examples**, which are collection of **features** that are qualitative or quantitative properties of the data that are exploited by the model to learn the task T .

2.3.1 The learning problem

A formal definition of a machine learning model is given below:

Definition 2. Let $\mathcal{X} \subseteq \mathbb{R}^d$ be the input space, and let p_{data} denote an unknown data-generating distribution over $\mathcal{X}$. We assume that the dataset consists of N independent and identically distributed (i.i.d.) samples drawn from p_{data} , namely

$$\mathcal{D}_{dev} = \{\mathbf{x}_i\}_{i=1}^N, \quad \mathbf{x}_i \sim p_{data}.$$

Let $\mathcal{Y}$ denote the output (or target) space. A **machine learning model** is a family of prediction functions

$$f_{\theta} : \mathcal{X} \rightarrow \mathcal{Y},$$

parameterized by $\theta \in \Theta$. The nature of the model and its learning objective are determined by the chosen learning paradigm:

- In the **supervised** setting, the data is drawn from a joint distribution $p_{data}(\mathbf{x}, y)$, so $\mathcal{D}_{train} = \{(\mathbf{x}_i, y_i)\}_{i=1}^k$, with targets $y_i \in \mathcal{Y}$. Here, $\mathcal{Y}$ (e.g., $\mathbb{R}$ for regression or $\{0, 1\}$ for classification), and the goal is to find parameters θ^* that minimize a loss function $\mathcal{L}(y, f(\mathbf{x}; \theta))$ quantifying the discrepancy between predictions and true labels.
- In the **unsupervised** setting, the data consists only of inputs $\mathbf{x}_i$. The output space $\mathcal{Y}$ is defined by the specific task (e.g., $\mathbb{R}^k$ for dimensionality reduction, a set of clusters for clustering, or $\mathbb{R}^d$ for generative models). The goal is to find parameters θ^* that optimize an objective function $\mathcal{J}(f(\mathbf{x}; \theta), \mathcal{D})$, which captures desired structural properties of the data, such as reconstruction error, cluster cohesion, or data likelihood.

In both cases, the ultimate objective is for the model $f(\mathbf{x}; \theta^*)$ to generalize well to unseen data $\mathcal{D}_{test} \sim P(\mathbf{x})$.

The model f_θ ² is said to belong to a **hypothesis space** $\mathcal{H}$, which is the set of all possible functions the model can represent by varying its parameters θ . The choice of model architecture (e.g., linear models, neural networks, decision trees) [17] defines the hypothesis space. A larger, more expressive $\mathcal{H}$ can represent more complex functions. The learning algorithm searches within $\mathcal{H}$ to find the hypothesis $f^* = f(\cdot; \theta^*)$ that best approximates the true underlying relationship between inputs and outputs.

2.3.2 Modelling approaches: Generative vs Discriminative

Both supervised and unsupervised models differ in how they represent data distributions. There are two main approaches: **Generative** and **Discriminative** learning. Discriminative models learn the conditional probability $p(y|\mathbf{x})$ directly, focusing on the decision boundary between classes. They are typically used for classification and regression tasks.

In contrast, generative models learn the joint probability distribution $p(\mathbf{x})$, modelling how the data is generated. So that they can generate samples that are similar to the training data, and are used for density estimation (e.g., Gaussian Mixture Models, Hidden Markov Models) [17, Ch. 5.8], unsupervised learning, and semi-supervised learning. How the parameters are estimated often³ reflects whether the approach is *Generative* or *Discriminative*.

Discriminative models inference is performed through **Maximum A Posteriori** (MAP) estimation, which seeks the best hypothesis (θ) given the data and prior knowledge. On the other hand, generative models are typically trained by a **Maximum Likelihood Estimation** (MLE), fitting the parameters that maximize the likelihood of the data.

In this work, we will focus mainly on discriminative approaches, even though we can still use our models for generative tasks.

2.3.3 Risk, Generalization and Overfitting

While the Bayesian perspective⁴ provides a probabilistic view for learning, in practice what is often adopted is called the *empirical* approach based on risk minimization [19]. This complements it by focusing on measuring and optimizing performance through the loss function, with the main goal of achieving *generalization*. We seek a function f_θ that minimizes the expected risk $\mathcal{R}(f_\theta)$ over $p_{\text{data}}(\mathbf{x}, y)$. The loss function acts as a proxy for the learning process. Risk has a dual nature: the expected risk and the empirical risk.

²We will use f_θ as shorthand for $f(\mathbf{x}; \theta)$

³However, this correspondence is not absolute [18, p. 312, Ch. 13]: logistic regression can also be trained by pure MLE, and generative models can incorporate priors and thus be estimated via MAP.

⁴As said in [18, p. 314, Ch. 13], Bayesian approach appears to be formally optimal, but a pragmatic approach is possible, it can be conditioned by a penalty λ on the complexity of the model

The **expected risk** (or population risk) of a hypothesis f_θ measures its average performance over the true data distribution. However, we do not have access to the entire p_{data} we can only observe a finite sample $\mathcal{D}_{\text{train}}$, therefore, we approximate the expected risk with the **empirical risk**: $\hat{\mathcal{R}}(f_\theta)$, which is the average loss over the training set.

The learning problem becomes finding θ^* that minimizes $\hat{\mathcal{R}}(f_\theta)$ over the training set. This is the **Empirical Risk Minimization** (ERM) principle, which forms the foundation of most practical machine learning algorithms.

Risk Decomposition

Most of the time, the space of possible solutions is unfathomably large; thus, we need to decompose the risk in order to understand if we are generalizing correctly.

Let $f_{\mathcal{H}}^* = \arg \min_{f \in \mathcal{H}} \mathcal{R}(f)$ be the best possible function within the hypothesis space, and let $\hat{f}_{\mathcal{H}} = \arg \min_{f \in \mathcal{H}} \hat{\mathcal{R}}(f)$ denote the empirical risk minimizer found by the learning algorithm. Then, the population risk of the learned model can be decomposed in: [20][19][21][22, pag. 33, Ch 2]

$$\mathcal{R}(f_\theta) = \underbrace{\mathcal{R}(f_\theta) - \hat{\mathcal{R}}(f_\theta)}_{\text{Generalization error}} + \underbrace{\hat{\mathcal{R}}(f_\theta) - \hat{\mathcal{R}}(f_{\mathcal{H}}^*)}_{\text{Optimization error}} + \underbrace{\hat{\mathcal{R}}(\hat{f}_{\mathcal{H}}) - \mathcal{R}(f_{\mathcal{H}}^*)}_{\text{Estimation error}} + \underbrace{\mathcal{R}(f_{\mathcal{H}}^*)}_{\text{Approximation error}} \quad (2.6)$$

The approximation error is irreducible because the data-generating process includes an intrinsic noise ε that cannot be captured by any hypothesis in $\mathcal{H}$. Consequently, learning algorithms must focus on reducing the remaining components. When the selected hypothesis f_θ is insufficiently expressive, **underfitting** occurs: the empirical risk $\hat{\mathcal{R}}(f_\theta)$ may be small, yet the estimated hypothesis fails to approximate the target function.⁵ If f_θ possesses excessive capacity, the estimator can *interpolate* $\mathcal{D}_{\text{train}}$, attaining negligible empirical risk, but with a large generalization gap; this phenomenon is called **overfitting**.

2.3.4 Optimizing and Regularizing

We extensively talked about different ways of learning, but yet we haven't discussed on how the learning is actually performed. Many of the modern ML algorithms are based on **Gradient-Based Optimization** methods [23], which iteratively update the model parameters θ in order to minimize the loss function $\mathcal{L}(y, f(\mathbf{x}; \theta))$. However, learning differs from pure optimization, ML methods optimize indirectly with respect to the population risk. The typical ML objective function minimized is:

$$\mathcal{L}(\theta) = \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}_{\text{train}}} [\mathcal{L}(y, f(\mathbf{x}; \theta))] + \lambda \Omega(\mathbf{w}) \quad (2.7)$$

⁵In the case of limited data availability

Questa è una parte
più pragmatica che
spiega come si fa
learning e come si
gestisce la complessità

Comunque nella
pratica se non facciamo
molto uso di gradi-
ent based optimiza-
tion, forse si può sor-
volare sui dettagli
serve comunque
una base background?

The term $\Omega(\mathbf{w})$ is a regularization term that allows to control the complexity of the model, λ is a hyperparameter that controls the strength of the regularization.

Regularization can be done in different ways, for example, using a prior distribution over the parameters, or in optimization, we can penalize with different norms, for example, $L2$ or $L1$ norms, if we are interested in decreasing the magnitude of parameters or inducing sparsity, respectively.

The algorithm that empowers the entire ML is **Stochastic Gradient Descent** (SGD) [24, pag. 152, Ch 5.2] which iteratively updates the parameters θ in the opposite direction of the gradient of the loss function with respect to the parameters. Typically, the parameters are initialised in some configuration θ_0 , a good initialization is often crucial to convergence and at each iteration t , we compute the gradient of the loss function with respect to the parameters $\nabla_{\theta} \mathcal{L}(\theta_t)$, then we update the parameters using a learning rate η that controls the step size or magnitude of update, and usually is one the most influential factors that determine how quickly and accurately the model converges in a region with an optimal solution.

The learning rule is written as:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t) \quad (2.8)$$

More advanced optimization algorithms exist, such as **Adam** [25] but are based on SGD with some modifications. Moreover, SGD can be performed in either online, mini-batch, or full-batch mode. In the online setting, the learning rule in (2.8) is applied using a single training example at a time. In contrast, mini-batch and full-batch SGD compute parameter updates using, respectively, a subset or the entirety of the training dataset $\mathcal{D}_{\text{train}}$. Mini-batch training is generally preferred when $|\mathcal{D}_{\text{train}}|$ is large, as processing the full dataset at each update step may be computationally infeasible or exceed available memory. Full-batch optimization is instead practical when the dataset is sufficiently small to fit entirely in memory. Mini-batch optimization introduces stochasticity into the parameter updates, resulting in noisy gradient estimates. This noise can be beneficial, as it may help the optimization process escape poor local minima or saddle points. At the same time, the expected update direction of mini-batch gradients approximates the true gradient of the empirical risk, as discussed in [26].

Model selection and Hyperparameters

The standard pipeline for training a machine learning model follows a systematic approach. Given a specific task and a sampled dataset $\mathcal{D}_{\text{dev}} \sim p_{\text{data}}$ partitioned into three subsets: training $\mathcal{D}_{\text{train}}$, validation $\mathcal{D}_{\text{val}}$, and test sets $\mathcal{D}_{\text{test}}$. The training set optimizes model parameters, the validation set serves for hyperparameter tuning and architectural decisions, and the test set provides an unbiased performance estimate. Hyperparameters are those parameters not learned during training

but set prior to the learning process. We have seen examples like the regularization coefficient λ and the learning rate η . The training phase primarily reduces the estimation error $\mathcal{R}(f_\theta) - \hat{\mathcal{R}}(f_\theta)$, from eq. ?? . Model selection, controls the approximation error $\mathcal{R}(f_{\mathcal{H}}^*)$ by determining the expressiveness of $\mathcal{H}$ through hyperparameter choices. Model selection involves training multiple candidate models with varying architectures and hyperparameters on the training set, then selecting the configuration with the best validation performance. Validation methods, such as K-fold cross-validation or Bootstrapping [17] and Bayesian optimization search [27] are often used in order to search more efficiently in the space of possible models.

2.4 Neural Networks and Deep Learning

Neural networks (NNs) [28, Ch. 3.1] are flexible non-linear universal function approximators [29] [22] which are inspired by the biological neural system, where information is processed in hierarchical manner. In fact, the idea originates from the layerwise brain organization in the visual cortex where information is processed at different complexity scales as discussed in [30]. Specifically, a shallow feedforward neural network (FFN) is composed of interconnected nodes called *perceptrons* [31] or artificial neurons divided into: input layer, that receives the perceived signal $\mathbf{x} \in \mathbb{R}^d$, one hidden layer, and an output layer that predicts the target $\hat{y}$. By stacking hidden layers we are composing together many different functions eg. $f_{net}^l \circ f_{net}^{l-1} \circ \dots \circ f_{net}^1$, with each hidden layer having its own parameters: $\theta = \{W^{(l)}, \mathbf{b}^{(l)}\}$, increasing the complexity of the model as well as the hypothesis space $\mathcal{H}$.

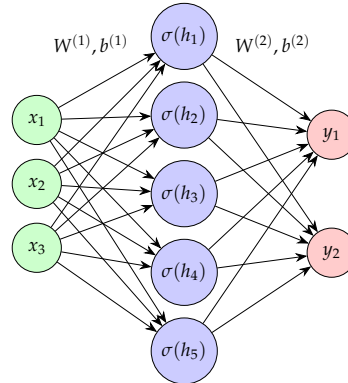


FIGURE 2.2: A simple Feedforward Neural Network with one hidden layer.

This is the main idea behind **Deep Learning** [28, Ch 4.3], which refers to Deep neural networks (DNNs) which are the core of many state-of-the-art machine learning systems [17].

Consider f_{net}^l to be the generic output at layer l of a DNN, composed by two parts: an affine transformation W , followed by a non-linear activation or transfer function σ .

$$f_{net}^l(\mathbf{x}) = \sigma(W^{(l)}\mathbf{x} + b^{(l)}) = \sigma\left(\sum_{i=1}^d w_i^{(l)} x_i + b^{(l)}\right)^6 \quad (2.9)$$

We call $W^{(l)}$ the weight matrix, $\mathbf{b}^{(l)}$ the bias vector, σ is an arbitrary⁷ non-linear function such as ReLU, Sigmoid or Tanh. $\mathbf{x} \in \mathbb{R}^d$ is the input vector. The parameters of this model, weight, are estimated through an algorithm known as **Backpropagation** [32], which is an application of the chain rule of calculus to compute the gradient of the loss function with respect to the model parameters.

2.4.1 Recurrent Neural Networks for Sequential Data

Recurrent Neural Networks (RNNs) [32] are a class of neural networks designed to handle sequential values. Their recurrent architecture introduces an *architectural bias* towards capturing sequential dependencies. Even prior to training, recurrent units can display clustering resembling Markovian dynamics [33].

In contrast to FNNs, which process input samples independently and assume a fixed-dimensional input representation, sequential data are characterized by temporal dependencies and long-range contextual structure. As a consequence, the effective complexity of modeling sequential data grows with the length of the sequence, i.e., with the number of time steps T .

Formally, RNNs are factorizing the probability of a sequence into a product of conditional distributions over time [34]. Rather than conditioning explicitly on the entire past sequence, RNNs introduce a hidden state that acts as a compact summary of previous observations. The probability of a sequence $\mathbf{x}_{1:T} = [\mathbf{x}_1, \dots, \mathbf{x}_T]$ is thus modeled as

$$p(\mathbf{x}_{1:T}) = \prod_{t=1}^T p(\mathbf{x}_t \mid \mathbf{h}_{t-1}; \theta), \quad (2.10)$$

where the hidden state $\mathbf{h}_t$ is updated recursively according to

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t; \theta). \quad (2.11)$$

Through this recursive update, the hidden state provides a finite-dimensional representation of the history $\mathbf{x}_{1:t}$, enabling the model to capture temporal dependencies without explicitly storing past inputs.

RNNs introduce parameter sharing across time steps, allowing them to generalize across different sequence lengths and capture temporal dependencies. Parameter sharing embodies a key *inductive bias*: the assumption of temporal stationarity. This assumption implies that the same parameters are updated each time step, so that

⁶With x_i and $w_i^{(l)}$ the i -th element of $\mathbf{x}$ and $W^{(l)}$ respectively

⁷While for hidden layers the right activation is usually chosen by model selection, for the output layer the activation regards the task e.g., identity for regression, logistic for binary classification, and softmax for multiclassifiers

the conditional probability distribution over variables at time $t + 1$ given all previous time steps does not depend explicitly on t , but only on the hidden state at time t .

For any time indices $t, k \in \mathbb{N}$, transition and output distributions: $p(\mathbf{h}_{t+1} \mid \mathbf{h}_t; \theta) = p(\mathbf{h}_{k+1} \mid \mathbf{h}_k; \theta)$ and $p(\mathbf{y}_t \mid \mathbf{h}_t; \theta) = p(\mathbf{y}_k \mid \mathbf{h}_k; \theta)$, share the same parameter set θ governing the dynamics at each time step, consequently, temporal dependencies are captured solely through the evolution of the hidden state $\mathbf{h}_t$, without explicit dependence on t .

Similar to seminal works as Time Delay Neural Networks (TDNNs) [35], RNNs maintain a hidden state $\mathbf{h}_t$ that is updated at each time step based on the current input $\mathbf{x}_t$ and the previous hidden state $\mathbf{h}_{t-1}$.

Computational graphs are often used to represent the structure of RNNs. These are part of the family of structured probabilistic models or **Graphical Models** [18], i.e., Hidden Markov Models (HMMs), Bayesian Networks (BNs), and Conditional Random Fields (CRFs) [18, Ch. 4]. Where nodes represent random variables, and edges represent dependencies between them, similarly, we can think to RNNs as a specialization of these probabilistic models, but with a parameterized approach to the dependencies.

An RNNs equation at time step t is defined as:

$$\mathbf{h}(t) = \tanh(W_{in}\mathbf{x}(t) + W_{rec}\mathbf{h}(t-1) + \mathbf{b}) \quad (2.12)$$

Where $\mathbf{h}(t) \in \mathbb{R}^{N_h}$ is the hidden state at time t , $\mathbf{x}(t) \in \mathbb{R}^{N_x}$ is the input vector at time t , $W_{in} \in \mathbb{R}^{N_h \times N_x}$ is the input weight matrix, $W_{rec} \in \mathbb{R}^{N_h \times N_h}$ is the recurrent weight matrix, $\mathbf{b} \in \mathbb{R}^{N_h}$ is the bias vector, and $\tanh$ is the hyperbolic tangent activation function applied element-wise.

The output $\mathbf{y}(t) \in \mathbb{R}^{N_y}$ at time t is computed as:

$$\mathbf{y}(t) = W_{out}\mathbf{h}(t) + \mathbf{c} \quad (2.13)$$

RNNs can be *unrolled* through time, which results in a directed acyclic graph (DAG), which helps visualizing the flow of information through such networks, as we can see from the figure ??.

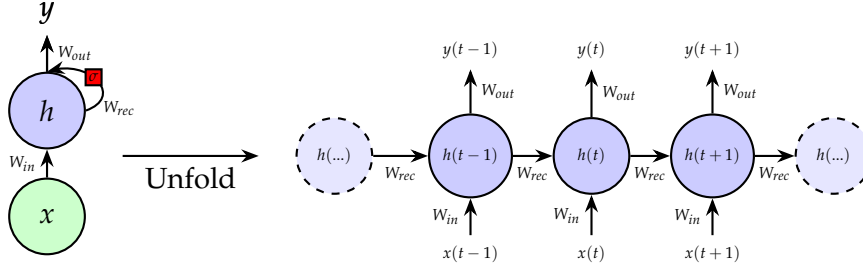


FIGURE 2.3: Vanilla Recurrent Neural Network unrolled through time: the hidden state for generic timestep t is updated with the same two matrices: W_{in} and W_{rec} by taking the previous hidden state h_{t-1} and the signal at the time t along the application of σ nonlinearity, the arrows acts as matrix-vector multiplications.

RNNs differ from traditional DNNs due to the sequential input nature; even training is performed in a different way, the algorithm used for training these models is called **Backpropagation Through Time** (BPTT) [36], which requires to backpropagate the error through from the end to the start of the unrolled graph ?? with $O(T)$ complexity.

2.4.2 The challenge for Long-Term Dependencies

RNNs are powerful models for sequential data modelling; however, in the last years, they have been largely replaced by Transformer architectures [3]. This happened mainly because of the difficulty in training RNNs to capture long-term dependencies in sequences [5], [37], [38]. The problem is addressed as *information latching* [39], which consists in the ability of such networks to retain a robust representation of bits of information, this happens when the states of the network are contained in a reduced attracting set of a hyperbolic attractor [40], in the neighbourhood of this set the derivatives vanish and hence cannot be propagated through time to reliably latch the set. In this work, we will try to address this problem when proposing our solutions that are based on previous literature on RNNs designed to capture long-term dependencies; we will briefly summarize the main issues and the solutions proposed in the literature.

The cause of Vanishing and Exploding Gradients

By looking again at picture ??, if we imagine a really long sequence, the gradients will be propagated back through many time steps, this can lead to numerical instability and make long-term dependencies hard to learn. This is due to the repeated

multiplication of the gradients by the Jacobian matrix of the hidden state with respect to itself at the previous time step. Essentially, we compute the following loss:

$$\frac{\partial \mathcal{L}_t}{\partial W_{rec}} = \sum_{k=1}^t \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \left(\prod_{l=k}^{t-1} \frac{\partial \mathbf{h}_{l+1}}{\partial \mathbf{h}_l} \right) \frac{\partial \mathbf{h}_k}{\partial W_{rec}} \quad (2.14)$$

where $\frac{\partial \mathbf{h}_{l+1}}{\partial \mathbf{h}_l}$ is the Jacobian matrix at time step l . The product $\prod_{l=k}^{t-1} \frac{\partial \mathbf{h}_{l+1}}{\partial \mathbf{h}_l}$ grows exponentially with the sequence length $(t - k)$, leading to *vanishing* or *exploding* (V/E) gradients⁸ [17, pag. 290, Ch 8.2] depending on the spectral properties of these Jacobian matrices. Suppose that W_{rec} eigendecomposes as $W_{rec}^t = V \text{diag}(\lambda)^t V^{-1}$. Exploding happen when the $|\lambda_{max}| > 1$, while vanishing when $|\lambda_{max}| < 1$. The Jacobian matrix provides information about how small changes in $\mathbf{h}_t$ propagate to the next step, or equivalently, how the gradient on $\mathbf{h}_{t+1}$ propagates at the previous step. Moreover, the recurrent matrix and the Jacobian can have complex-valued eigenvalues, with an imaginary part that induces oscillatory dynamics, making more complicated dynamics easier to learn.

Several solutions have been proposed in literature to mitigate this problem, such as gradient clipping [41], residual connections [42] that help by adding to the timesteps directly the signal from previous steps, as well approaches directly acting on the recurrent matrix that fixes its spectrum to 1, such as orthogonal or unitary RNNs [43] [44]. Even choosing the right transfer function can help: Sigmoidal activations tend to saturate very quickly since their derivatives approach 0 in the extreme regions, alternatives such as tanh that saturate less, or ReLU that has more stable gradients and regularize at the same time by sparsifying activations [45] contribute to the architectural bias.

A fundamental insight into solving the vanishing gradient came from Hochreiter et al. [5], introducing the concept of **Constant Error Carousel** (CEC), which allows gradients to flow unchanged through time in the backward pass by using self-loops with unit weight in the recurrent connections. But with CEC, the error is integrated over time, and time-based tasks often require memory to change dynamically.

Gated RNNs

To address the memory issues of learning long-term dependencies in sequential data, specialized RNN architectures with gating mechanisms have been developed, specifically Long Short-Term Memory (LSTM) networks [5] and Gated Recurrent Units (GRUs) [46]. Such architectures aimed to solve Neural Machine Translation (NMT) over long sentences, where standard RNNs failed due to saturation of their memory capabilities. The gates in these models solve the CEC problem by allowing the network to learn selectively what information should be integrated or forgotten over time.

⁸Gradient issues is related to depth of a network, even sufficiently deep FFNs suffers of V/E issues.

The *forget gate* uses sigmoid activation to decide what amount of information to discard from the previous h_t , while the *input gate* controls how much of the new information from the current input x_t should be added to the cell state. Finally, the *output gate* determines what part of the cell state should be output as the new hidden state h_t .

For LSTMs the equations governing the gates and cell state updates are as follows:

$$\begin{aligned}
 f_t &= \sigma(W_f \cdot [h_{t-1}, x_t]) \quad (\text{forget gate}) \\
 i_t &= \sigma(W_i \cdot [h_{t-1}, x_t]) \quad (\text{input gate}) \\
 o_t &= \sigma(W_o \cdot [h_{t-1}, x_t]) \quad (\text{output gate}) \\
 \tilde{C}_t &= \tanh(W_C \cdot [h_{t-1}, x_t]) \quad (\text{cell candidate}) \\
 C_t &= f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (\text{cell state update}) \\
 h_t &= o_t \odot \tanh(C_t) \quad (\text{hidden state update})
 \end{aligned} \tag{2.15}$$

Where W_f, W_i, W_o, W_C are weight matrices for the respective gates and cell candidate.

Instead, the GRU architecture combines the forget and input gates into a single *update gate*, simplifying the model while retaining similar capabilities. But having fewer parameters makes GRUs faster to train and less prone to overfitting on smaller datasets. **Update gates** acts as a conditional leaky integrator, deciding how much of previous hidden state to retain, while the **reset gate** determines how much of previous hidden state h_{t-1} to forget and how much of the new input x_t to consider when computing the candidate hidden state $\tilde{h}_t$ by using a convex between the two. The GRU equations are as follows:

$$\begin{aligned}
 z_t &= \sigma(W_z \cdot [h_{t-1}, x_t]) \quad (\text{update gate}) \\
 r_t &= \sigma(W_r \cdot [h_{t-1}, x_t]) \quad (\text{reset gate}) \\
 \tilde{h}_t &= \tanh(W_h \cdot [r_t \odot h_{t-1}, x_t]) \quad (\text{candidate hidden state}) \\
 h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (\text{hidden state update})
 \end{aligned} \tag{2.16}$$

Where W_z, W_r, W_h are weight matrices for the respective gates and candidate hidden state.

va bene cosi per la formula o devo aggiungere cosa significa ogni parte?

Both LSTMs and GRUs have demonstrated remarkable success in various sequential tasks, including language modelling. But they still rely on BPTT for training, inheriting its limitations regarding computational cost and difficulty in capturing very long-term dependencies.

2.5 Deep Recurrent Neural Networks

The concept of *depth* on RNNs is ambiguous and can be extended in multiple ways, as discussed by [47]. Firstly, we say that a deep network applies multiple non-linear transformations to the input data; this is trivial for FNNs, but in the case of vanilla⁹ RNNs, the depth is embedded into the temporal structure, since we apply at each time step the same non-linearity. In the picture ?? we can see that there are several ways to introduce depth in RNNs.

If we implement the depth from the *input-to-hidden* layers, the networks improve at disentangling the underlying factors of variations from the original input signal, similar to how Convolutional layers in Convolutional Neural Networks (CNNs) extract features hierarchically [48]. For *hidden-to-output* intermediate layers instead, the networks learn a more compact latent representation, resulting in a more efficient summarization of input history into the hidden state that leads to the output layer, so the readout is simplified.

Yet another way to introduce depth is the *hidden-to-hidden* transition, which essentially allows the hidden state to adapt to mode shifts in the input signal rapidly, this could be particularly useful in non-stationary environments, however, a deep transition approach could lead to optimization issues due to the increased depth in the temporal dimension.

Finally, we can stack multiple recurrent layers on top of each other, which is the main drive of this thesis, this approach encourages the model to learn at different timescales with the first layers capturing fast dynamics and the deeper layers capturing slower ones.

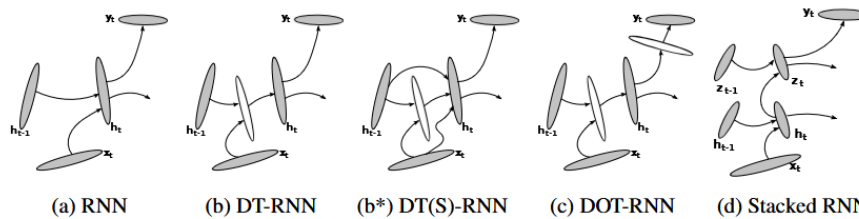


Figure 2: Illustrations of four different recurrent neural networks (RNN). (a) A conventional RNN. (b) Deep Transition (DT) RNN. (b*) DT-RNN with shortcut connections (c) Deep Transition, Deep Output (DOT) RNN. (d) Stacked RNN

FIGURE 2.4: Depths in Recurrent Neural Networks represented graphically by [47]

2.6 Reservoir Computing

Reservoir Computing (RC) is a computational framework introduced in the early 2000s with two key models: Echo State Networks (ESN) [49] and Liquid State Machines (LSM) [50]. Both approaches aim to simplify the training of RNNs while

⁹In literature "vanilla" term refers to non-gated RNNs as the equation ?? describes

exploiting the inherent dynamical properties of real-world physical systems to obtain a "reservoir" of rich dynamics that can be leveraged to solve complex tasks. A major advantage of RC lies in its efficiency: only the linear readout layer is trained, while recurrent connections remain fixed.

The central idea is to avoid training the recurrent connection in the network by using a fixed high-dimensional reservoir that projects the input signal into a high-dimensional feature space, where a linear classifier is sufficient to learn the linearized representation of the input signal.

Reservoir Computing has been successfully applied to tasks such as time series prediction, speech recognition, robot control, furthermore can also be extended to generative settings using techniques such as Teacher Forcing [51].

add citation here

The focus of this work is on RC methods that address the challenge of solving learning long-term dependencies, by directly avoiding the use of BPTT and its issues. From a theoretical viewpoint, RC models can be viewed as generalization of **kernel methods** [40, Ch. 5] to sequential data, these type of models solve the task in two steps: firstly they project the input data into a high dimensional space using a fixed non-linear mapping (the reservoir in RC), secondly they learn a linear model on top of this representation to make predictions.

Both RC and kernel methods relies on the **Cover's Theorem** [52], which states :

"A complex pattern-classification problem, cast in a high-dimensional space nonlinearly, is more likely to be linearly separable than in a low-dimensional space."

2.6.1 Echo State Networks

ESNs are a specific type of fully connected RNN, where only a small part of recurrent connections in the hidden layer is active, inducing sparsity, and only the output weights are trained. A common approach to build rich reservoirs [17, Ch 10.8] is to design the recurrence within a state-space system that operates in the *edge of chaos* regime as described by [53] [54]. The idea is to constrain the eigenvalues of the system's Jacobian matrix $\mathbf{J}$ to be close to 1. As we previously mentioned in sec ??, we want to enforce this condition on: $\mathbf{J}_t = \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}}$.

In practice, this is controlled by the **spectral radius** of such $\mathbf{J}$, defined as the largest absolute eigenvalue of the matrix, denoted as $\rho(\mathbf{J}) = \max |\lambda_i|$. By constraining $\rho(\mathbf{J})$ to be close to 1, we can ensure that the system is in the edge of chaos regime, which is characterized by a balance between stability and instability.

In ESNs we can control the spectral radius of the recurrent weight matrix $\mathbf{W}_{rec} \in \mathbb{R}^{N_r \times N_r}$ directly during initialization by scaling it appropriately. Given a randomly initialized weight matrix $\mathbf{W}_{rec}^{init}$, we can compute its spectral radius $\rho(\mathbf{W}_{rec}^{init})$ and then

scale it to a desired value $\rho_{desired}$:

$$W_{rec} = \frac{\rho_{desired}}{\rho(W_{rec}^{init})} W_{rec}^{init} \quad (2.17)$$

Even the $W_{in} \in \mathbb{R}^{N_r \times N_u}$ matrix is usually scaled by an important hyperparameter called **input scaling** ω_{in} , which controls the influence of the input signal on the reservoir dynamics. This parameter is crucial because it affects how the reservoir responds to different input magnitudes and can impact the overall performance of the ESN. A common practice is to scale W_{in} as follows:

$$W_{in} = \omega_{in} W_{in}^{init} \quad (2.18)$$

A more general formulation is the Leaky ESN is given by [55], which introduces a leaking rate $\alpha \in [0, 1]$ to control the speed of the reservoir dynamics:

$$\mathbf{h}(t) = (1 - \alpha)\mathbf{h}(t - 1) + \alpha \tanh(W_{in}\mathbf{u}(t) + W_{rec}\mathbf{h}(t - 1) + \mathbf{b}) \quad (2.19)$$

When $\alpha = 1$, the Leaky ESN reduces to the standard ESN, while for $\alpha < 1$, the reservoir state updates more slowly, allowing it to capture longer-term dependencies in the input sequence.

Tikhonov Regularization

Once the reservoir is initialized, we can use it to process input sequences and collect the reservoir states over time. After we computed the reservoir states $\mathbf{h}(t)$ for the entire input sequence, we have to find W_{out} that maps the reservoir states to the desired output.

Given a set of target outputs $\mathbf{y}(t)$ corresponding to each input $\mathbf{x}(t)$, our objective is to minimize the squared norm of the difference between the actual outputs and the predicted outputs from the reservoir states:

$$\mathcal{L} = \sum_{t=1}^T \|\mathbf{y}(t) - W_{out}\mathbf{h}(t)\|^2 \quad (2.20)$$

We can train the output weights W_{out} using ridge regression in closed form as:

$$W_{out} = YH^T(HH^T + \lambda I)^{-1} \quad (2.21)$$

Where $Y \in \mathbb{R}^{N_y \times T}$ is the matrix of target outputs, $H \in \mathbb{R}^{N_r \times T}$ is the matrix of reservoir states collected over time, λ is the regularization parameter, and I is the identity matrix.

To prevent overfitting on readout layer, and since the matrix H is often ill-conditioned, we use regularization hyperparameter λ [56] during the training of the output weights. What this does is essentially to keep the diagonal of the matrix HH^T away from zero, improving its condition number and making the inversion numerically stable.

Transients dynamics

It is also possible that random effects from the initialization of the reservoir can lead to suboptimal performance; hence, usually the initial timesteps collected by the reservoir are discarded, because the reservoir undergoes a *transient dynamics* phase before the state becomes fully dependent on input history rather than initial conditions. In literature, this phase is called **Washout period** T_{washout} . Given the corresponding reservoir states $\{\mathbf{h}(t)\}_{t=1}^T$, only the states $\{\mathbf{h}(t)\}_{t=T_{\text{washout}}+1}^T$ are used for training the readout layer, the washout period T_{washout} is empirically determined, usually hundreds or thousands of timesteps are sufficient.

2.6.2 Separation Property

Besides stability, a useful reservoir should also be able to *distinguish* different input histories. In the Reservoir Computing literature (especially for Liquid State Machines), this intuition is captured by the **separation property** (also called **pointwise separation**) [maass2002realtime]. Informally, the reservoir should map different input streams to different internal states, so that a linear readout can discriminate them. More precisely, for two bounded input sequences $\mathbf{x}(t)$ and $\tilde{\mathbf{x}}(t)$ that differ at least at one time step, the corresponding reservoir state trajectories $\mathbf{h}(t)$ and $\tilde{\mathbf{h}}(t)$ should become different at some time:

$$\exists t \geq 0 : \mathbf{h}(t) \neq \tilde{\mathbf{h}}(t) \quad (2.22)$$

This property is closely related to the common RC intuition that the reservoir acts as a high-dimensional non-linear expansion of the input (cf. ??), while the readout performs the final linear separation.

2.6.3 Echo State Property

While separation ensures that different inputs can lead to different internal representations, RC models also require the reservoir dynamics to be *stable* so that states do not depend on arbitrary initial conditions. The **Echo State Property** (ESP) [49] formalizes this requirement, stating that for any bounded input sequence, the reservoir state should asymptotically depend only on the input history and not on the initial state.

Formally, let $\mathbf{x}(t)$ be a bounded input sequence and $\mathbf{h}_1(0)$, $\mathbf{h}_2(0)$ be two different initial reservoir states obtained by running that sequence through the reservoir. The

ESP requires that:

$$\lim_{t \rightarrow \infty} \|\mathbf{h}_1(t) - \mathbf{h}_2(t)\| \rightarrow 0 \quad (2.23)$$

When this condition holds, the model behaves as a *fading memory* filter, where the influence of past inputs diminishes over time, in DS theory these are also called *contractive systems* [13].

Definition 3. A function $f : \mathbb{R}^N \times \mathbb{R}^M \rightarrow \mathbb{R}^N$ is **contractive** if there exists a constant $0 \leq k < 1$ such that for all $\mathbf{h}_1, \mathbf{h}_2 \in \mathbb{R}^N$ and $\mathbf{x} \in \mathbb{R}^M$:

$$\|f(\mathbf{h}_1, \mathbf{x}) - f(\mathbf{h}_2, \mathbf{x})\| \leq k \|\mathbf{h}_1 - \mathbf{h}_2\| \quad (2.24)$$

And k is called the **Lipschitz constant** of the function f .

For it to be true, we must impose two conditions, a necessary and a sufficient one, both regarding the spectral properties of the recurrent weight matrix W_{rec} . The necessary condition requires:

$$\|\rho(W_{rec})\| < 1 \quad (2.25)$$

While the sufficient condition requires:

$$\|\sigma_{max}(W_{rec})\| < 1 \quad (2.26)$$

Where $\rho(W_{rec})$ is the spectral radius of the matrix W_{rec} and $\sigma_{max}(W_{rec})$ is the largest singular value of the matrix W_{rec} , recall from [57][pp. 347], that the spectral radius is always upper bounded by the largest singular value: $\rho(W_{rec}) \leq \sigma_{max}(W_{rec}) = \|W_{rec}\|_2$.

Although ESP guarantees that the reservoir state depends uniquely on the input history and not on initial conditions, enforcing its sufficient condition, $\rho(W_{rec}) < 1$, is often too restrictive in practice. This bound ensures that the recurrent dynamics are contractive even in the absence of input, thereby guaranteeing stability under all circumstances. However, it also leads to reservoirs with overly damped dynamics, where perturbations decay too quickly, and the network loses its ability to store temporal information. The sufficient conditions are derived under the assumption of an autonomous system, neglecting the stabilizing effect of the continuous external input. Empirically, the most effective reservoirs operate near the edge of chaos, and the spectral radius is often set even slightly above 1 in some cases [58].

Where $u(t)$ is the input signal at time t , $y_k(t)$ is the reservoir output trained to predict the input delayed by k time steps, and $corr$ denotes the Pearson correlation coefficient.

The MC provides a measure of how well the reservoir can reconstruct past inputs from its current state, indicating its capacity to retain temporal information.

Jeager has also shown a theoretical bound for MC [59], that the MC of a Linear read-out RNN with n neurons trained to reconstruct a delayed version of an i.i.d input process, is bounded by n .

2.7 Deep Echo State Networks

In the research field of Reservoir Computing, the concept of depth has been explored to enhance the representational capacity of reservoirs. By stacking multiple recurrent layers, the models can capture hierarchical temporal features at different timescales [47], and improve performance on complex tasks requiring multi-scale processing or longer memory retention.

In [60] [61] the authors focus on the analysis of Deep Echo State Networks (DeepESN). Having multiple sequentially stacked layers where information can be processed, allows for a different hidden state for each of them. The signal is given to the first sub-reservoir and then propagated through the stack, altering intrinsic dynamics and allowing a more expressive architectural bias. Experiments have also shown an increase in memory capability, thus DeepESNs are able to retain more information about past inputs compared to shallow ESNs with the same number of total neurons.

We define a Deep Echo State Network (DeepESN), with leaky integrator neurons, as a stack of L recurrent reservoir layers, where each layer l has its own state vector $\mathbf{h}^{(l)}(t)$. Each layer can have a different leaking rate, input scaling (from the previous layer) and spectral radius. The input to the first layer is the external input $\mathbf{x}(t)$, while the input to subsequent layers is the state of the previous layer.

The state update equations for a DeepESN are given by:

$$\mathbf{h}^{(l)}(t) = (1 - \alpha^{(l)})\mathbf{h}^{(l)}(t-1) + \alpha^{(l)} \tanh(W_{in}^{(l)} \mathbf{x}(t) + W_{rec}^{(l)} \mathbf{h}^{(l)}(t-1) + \mathbf{b}^{(l)}) \quad (2.27)$$

With $W_{in}^{(1)} \in \mathbb{R}^{N_r \times N_u}$, when $l \neq 1$ $W_{in}^{(l)} \in \mathbb{R}^{N_r \times N_r}$ the input weights matrix from previous layer, $\alpha^{(l)} \in [0, 1]$ the leaking rate, $W_{rec}^{(l)} \in \mathbb{R}^{N_r \times N_r}$ the recurrent weights matrix, $\mathbf{b}^{(l)} \in \mathbb{R}^{N_r}$ the bias vector.

For the readout phase, we can use the concatenated states from all layers or just the last layer:

$$\begin{cases} \mathbf{y}(t) = W_{out}[\mathbf{h}^{(1)}(t); \mathbf{h}^{(2)}(t); \dots; \mathbf{h}^{(L)}(t)] & \text{(concatenated states)} \\ \mathbf{y}(t) = W_{out}\mathbf{h}^{(L)}(t) & \text{(last layer state only)} \end{cases} \quad (2.28)$$

2.7.1 Echo state property of DeepESN

ESNs are designed to satisfy the Echo State Property (ESP) [49], which ensures that the reservoir's state is uniquely determined by the input history. When extending ESNs to DeepESNs, it is crucial to analyse how the ESP is affected by the depth of the network. In [61], the authors provide sufficient and necessary conditions for the ESP to hold in DeepESNs. They show that the ESP can be guaranteed if each layer of such a model satisfies the ESP individually, and if the inter-layer connections are appropriately scaled. Specifically, they derive conditions on the spectral radius of

the recurrent weight matrices and the norms of the inter-layer weight matrices to ensure stability and the ESP.

Formally, for a DeepESN with L layers, where each layer l is governed by a leaking rate $a^{(l)}$, a recurrent weight matrix $\mathbf{W}^{(l)}$, and receives input from the previous layer via $\mathbf{W}_{in}^{(l)}$, the conditions are defined as follows.

Necessary Condition The necessary condition for the asymptotic stability of the zero state (and thus for the ESP) relies on the analysis of the system's Jacobian. Due to the hierarchical, feedforward architecture of a standard DeepESN, the Jacobian is a block lower triangular matrix. Consequently, its global spectral radius is determined by the maximum of the spectral radii of the individual layers. The condition is given by:

$$\rho(\mathbf{J}_{DeepESN,x}(0_u, 0)) = \max_{l=1,\dots,L} \left(\rho \left((1 - a^{(l)})\mathbf{I} + a^{(l)}\mathbf{W}_{rec}^{(l)} \right) \right) < 1, \quad (2.29)$$

where $\rho(\cdot)$ denotes the spectral radius and $\mathbf{I}$, 0_u is the zero input, and 0 is the zero state of the DeepESN. $\mathbf{I}$ is the identity matrix. This implies that for the network to be stable around the zero state, every layer must individually satisfy the local stability criterion.

Sufficient Condition The sufficient condition requires the global dynamics of the DeepESN to be contractive. This is ensured if the maximum Lipschitz constant among all layers, denoted as $C^{(l)}$, is less than 1. The contraction coefficients are defined recursively to account for the propagation of signals through the depth of the network:

$$\begin{cases} C^{(1)} = (1 - a^{(1)}) + a^{(1)}\|\mathbf{W}_{rec}^{(1)}\| \\ C^{(l)} = (1 - a^{(l)}) + a^{(l)} \left(C^{(l-1)}\|\mathbf{W}_{in}^{(l)}\| + \|\mathbf{W}_{rec}^{(l)}\| \right) \end{cases} \quad \text{for } l > 1, \quad (2.30)$$

where $\|\cdot\|$ denotes the induced matrix norm (e.g., the spectral norm). The ESP is guaranteed if:

$$\max_{l=1,\dots,L} C^{(l)} < 1. \quad (2.31)$$

This indicates that in deeper layers, the contractivity depends not only on the local recurrent weights $\mathbf{W}_{rec}^{(l)}$ but also on the contraction of the previous layer $C^{(l-1)}$ and the magnitude of the inter-layer weights $\mathbf{W}_{in}^{(l)}$.

Because the recursive term involves $C^{(l-1)}$, the contraction coefficient increases with depth, making deeper layers less contractive and able to retain information longer.

2.8 Simple Cycle Reservoirs

In [62], the authors tackle reservoir design from a minimalistic perspective, the main question from the authors is:

"What is the minimal complexity of reservoir construction for obtaining competitive models and what is the memory capacity of such simplified reservoirs?"

Instead of using large, randomly initialized reservoirs, they propose much simpler topologies such as **Simple Cycle Reservoirs** (SCRs), **Delay Line Reservoirs** (DLRs) and **Delay Line Reservoirs with feedback connection** (DLRBs), by just connecting single units in a cycle or in a line with feedback from one layer to the next and vice versa.

The SCR is constructed by connecting N_r neurons in a unidirectional ring, where each neuron is connected to the next one in the cycle, and the last neuron connects back to the first neuron. The recurrent weight matrix W_{rec} for an SCR has the following structure:

$$W_{rec} = \begin{bmatrix} 0 & 0 & \cdots & 0 & w \\ w & 0 & \cdots & 0 & 0 \\ 0 & w & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & w & 0 \end{bmatrix} \quad (2.32)$$

Where w is the weight of the connections between neurons in the cycle.

Despite its simplicity, the SCR topology is competitive with traditional reservoir methods at least for linear reservoirs, thus justifying a well-structured topology can have a significant impact on the reservoir's dynamics and performance.

The authors also proved a bound for SCR on its memory capacity to be arbitrary close to the number of units N_r by properly tuning the input scaling and the recurrent weight w . With the following bound:

$$MC = N - (1 - r^{2N}) \quad (2.33)$$

dal paper dicono che su alcuni dataset come NARMA avevano bisogno di reservoir non linear

Qui dovrò metterci una ref alla part in cui parlo di MC nel capitolo successivo

2.9 Antisymmetric RNNs

Another approach to mitigating the vanishing and exploding gradient problem in recurrent neural networks is the *AntisymmetricRNN* proposed by [63].

The method is based on interpreting RNNs as continuous-time dynamical systems and designing the recurrent weight matrix according to stability principles from ODE theory.

The core idea is to constrain the recurrent matrix to be **antisymmetric**, because anti-symmetric matrices have purely imaginary eigenvalues.

$$W_{rec} = -W_{rec}^T \quad (2.34)$$

This property ensures that the continuous-time system is **stable**, meaning that small perturbations in the hidden state do not grow unbounded over time.

Since only being stable is not sufficient to capture long-term dependencies, the authors argue that if the system is dissipative, then using such an ODE would result in catastrophic forgetting of past inputs in the forward pass. A possible solution is to design a system that operates in a critical regime, such idea is formalized through the following **critical criterion**:

$$\max_i \operatorname{Re}(\lambda_i(J(t))) \approx 0, \quad \forall t > 0, \quad (2.35)$$

meaning that the real parts of the Jacobian eigenvalues remain close to zero. In this regime, long-term dependencies are preserved without inducing instability.

The proposed ODE system is defined as:

$$h'(t) = \tanh((W_{rec} - W_{rec}^T)h(t) + W_{in}x(t) + b) \quad (2.36)$$

If we derive the Jacobian of the system, we obtain:

$$J(t) = \operatorname{diag}[\tanh'((W_{rec} - W_{rec}^T)h(t) + W_{in}x(t) + b)](W_{rec} - W_{rec}^T) \quad (2.37)$$

Since the eigenvalues of an antisymmetric matrix are purely imaginary, and the derivative of the tanh activation is bounded between 0 and 1, the eigenvalues of the Jacobian $J(t)$ will have real parts close to zero, satisfying the critical criterion.

This condition for a discretized version of the ODE is very similar to the edge of chaos condition discussed previously for ESNs in sec ??, where the spectral radius of the Jacobian is constrained to be close to 1. To ensure stability after discretization with forward Euler method with timestep ϵ , the authors introduce a small diffusion term $-\gamma I$, since the eigenvalues of the Jacobian matrix: $|1 + \epsilon\lambda_i(J_t)| > 1$, leading to

the following update equation for the hidden state:

$$h(t) = h(t-1) + \epsilon \tanh((W_{rec} - W_{rec}^T - \gamma I)h(t-1) + W_{in}x(t) + b_h) \quad (2.38)$$

2.10 Random Oscillator Networks

Random Oscillator Networks (RONs) [64] are a class of physically-inspired reservoir computing models that drive the input signal through a network of heterogeneous oscillators, this model for chaotic time series prediction tasks outperformed other randomized approaches on long-term memory for sequence classification. The idea behind RONs is to leverage the rich dynamics of coupled oscillators to create a reservoir that, even on a small parameterization regime, can achieve high performance on temporal tasks.

Physical Dissipation and Fading Memory

In RONs, we get the *fading memory* property in the energy dissipation of damped oscillators. Considering the continuous-time energy function of a single unforced oscillator $\mathcal{E}(t) = \frac{1}{2}\dot{y}^2 + \frac{1}{2}\gamma y^2$, its time derivative is given by:

$$\dot{\mathcal{E}}(t) = -\epsilon \dot{y}^2. \quad (2.39)$$

For a strictly positive damping coefficient $\epsilon > 0$, the energy is strictly decreasing ($\dot{\mathcal{E}} \leq 0$), ensuring that the system asymptotically forgets its initial state transient.

The RON is defined as a discretized system of coupled Ordinary Differential Equations (ODEs). Let $\mathbf{y}_k \in \mathbb{R}^N$ and $\mathbf{z}_k \in \mathbb{R}^N$ represent the position and velocity vectors of the N oscillators at time step k . The state update is governed by explicit Euler discretization with step size τ :

$$\begin{aligned} \mathbf{y}_{k+1} &= \mathbf{y}_k + \tau \mathbf{z}_{k+1}, \\ \mathbf{z}_{k+1} &= \mathbf{z}_k + \tau [\tanh(\mathbf{W}\mathbf{y}_k + \mathbf{V}\mathbf{u}_{k+1} + \mathbf{b}) - \gamma \odot \mathbf{y}_k - \epsilon \odot \mathbf{z}_k]. \end{aligned} \quad (2.40)$$

Here, $\gamma, \epsilon \in \mathbb{R}^N$ are the heterogeneous stiffness and damping coefficients, respectively, and $\odot$ denotes the Hadamard product. The term $\tanh(\cdot)$ acts as a non-linear forcing function, coupling the oscillators through the recurrent weight matrix $\mathbf{W}$.

Jacobian and Stability Analysis

To analyze the stability and the spectral properties of the reservoir, we consider the Jacobian $\mathbf{J}_k$ of the state mapping $\mathcal{F}(\mathbf{y}_k, \mathbf{z}_k)$. By linearizing Eq. (??), the Jacobian exhibits a specific block structure:

$$\mathbf{J}_k = \begin{bmatrix} \frac{\partial \mathbf{y}_{k+1}}{\partial \mathbf{y}_k} & \frac{\partial \mathbf{y}_{k+1}}{\partial \mathbf{z}_k} \\ \frac{\partial \mathbf{z}_{k+1}}{\partial \mathbf{y}_k} & \frac{\partial \mathbf{z}_{k+1}}{\partial \mathbf{z}_k} \end{bmatrix} = \begin{bmatrix} \mathbf{I} + \tau^2 \mathbf{A}_k & \tau(\mathbf{I} - \tau \text{diag}(\epsilon)) \\ \tau \mathbf{A}_k & \mathbf{I} - \tau \text{diag}(\epsilon) \end{bmatrix}, \quad (2.41)$$

where $\mathbf{A}_k = \mathbf{S}_k \mathbf{W} - \text{diag}(\gamma)$, and $\mathbf{S}_k$ is the diagonal matrix of derivatives of the activation function, $\mathbf{S}_k = \text{diag}(\mathbf{1} - \tanh^2(\dots))$.

This block structure reveals that the eigenvalues of $\mathbf{J}_k$ are clustered in disks centered at $1 - \tau\epsilon$ and $1 - \tau^2\gamma$.

By tuning these hyperparameters, we can constrain the spectrum within the unit circle, facilitating the *edge of stability* regime required for long-term memory tasks. For small values of τ , the eigenvalues are close to 1, promoting slow decay of information and enhancing memory retention.

2.11 Euler State Network

Chapter 3

Methodology

After having discussed all the background concepts and related works, we can now move on to the methodology used in this work. In this chapter we present the idea to treat deep recurrent neural network as *modular deep reservoir* architectures. Rather than treating reservoir as a monolithic recurrent system, we interpret each layer of the deep reservoir as an independent computational module that processes inputs and exchanges information with other modules through structured interconnections.

3.1 Modularity in RNNs

In dynamical systems theory, a system is said to be *modular* if it can be decomposed into distinct subsystems (modules) each of which has its own internal dynamics while also interacting with other modules. Such composition allows for a complex global behaviour to emerge from the interactions of simpler components. At the same time, analyzing the system in terms of local module dynamics can provide control on information flow and interpretability benefits.

Modularity has been extensively studied in neuroscience, where large-scale brain networks exhibit modular organization with specialized regions that acts as hubs for information integration. Empirical studies from [65] have shown that neural systems are composed of densely connected subnetworks that interact globally as a sparse system, supporting both integration of information and specialized processing

More generally, studies in complex systems and network dynamics have shown that modular organization can enhance robustness by inducing a separation of time scales, such that dynamical processes remain confined within subsystems for extended periods before spreading globally. This structure allows perturbations to decay locally and supports stable large-scale dynamics under appropriate stability conditions [66].

finire di scrivere
questo quando avrò
un'idea di come sarà
fatta la metodologia

che sono connesse in
modo sparso forse è
meglio?

Forse quando parlo
in questo modo della
modularità nella tesi
dovrei anche dire
che stiamo trattando
le architetture deep
"normali" come se
fossero moduli o è
già sottinteso?

3.2 Deep Modular Reservoirs

We model a deep reservoir as a composition of L layers or modules. Each module $l \in \{1, \dots, L\}$ has state $\mathbf{h}^{(l)}(t) \in \mathbb{R}^{n_l}$ and internal recurrent connectivity $\mathbf{W}^{(l)}$. The key methodological contribution is to compare two architectures of deep modular reservoirs:

- **Deep Cycle Reservoir** where we employ *cycle/ring* recurrent topology, inspired by Simple Cycle Reservoirs (SCR), to provide strong memory traces with minimal parameters.
- **Antisymmetric deep reservoir** with *forward and backward* coupling between adjacent modules, designed to promote stable dynamics and improve gradient-free memory propagation.

3.2.1 Global state and dynamics

Let $\mathbf{H}(t)$ be the global reservoir state at time t obtained by feeding the input $\mathbf{u}(t)$ into the first module and propagating through the layers. By stacking the states of all modules, we have the final feature vector used for the readout is made

$$\mathbf{H}(t) = [\mathbf{h}^{(1)}(t), \mathbf{h}^{(2)}(t), \mathbf{h}^{(L)}(t)] \in \mathbb{R}^N; \quad N = \sum_{l=1}^L n_l. \quad (3.1)$$

Where N is the total reservoir size and n_l is the size of module l . The global dynamics of the reservoir can be written in the compact form: obtained after the application of the activation function and leaky integration.

We define the pre-activation vector

$$\mathbf{z}(t+1) = \mathbf{W}_{\text{tot}}\mathbf{H}(t) + \mathbf{W}_{\text{in}}\mathbf{u}(t+1)^1 + \mathbf{b}. \quad (3.2)$$

The reservoir state is then updated as

$$\mathbf{H}(t+1) = (1 - \alpha)\mathbf{H}(t) + \alpha \tanh(\mathbf{z}(t+1)). \quad (3.3)$$

$$\mathbf{H}(t+1) = \tanh(\mathbf{W}_{\text{tot}}\mathbf{H}(t)), \quad (3.4)$$

where $\tanh(\cdot)$ acts element-wise on the state vector and $\mathbf{W}_{\text{tot}} \in \mathbb{R}^{N \times N}$ encodes both intra-module and inter-module connections. $\mathbf{W}_{\text{tot}} \in \mathbb{R}^{N \times N}$ is a block matrix that encodes both intra-module and inter-module connections. For a simple Leaky-ESN with 1 layer, the $\mathbf{W}_{\text{tot}}$ is simply the recurrent weight matrix $\mathbf{W}_{\text{rec}}$ of the reservoir.

Finally, the structure of $\mathbf{W}_{\text{tot}}$ depends on the chosen architecture, a randomly connected module architecture would have a block diagonal structure, plus random connections between modules.

¹This input term is just for the input not the inter-layer connection

DeepESN as modular system

In the paper from [60] the authors propose to study what happens when treating modules as a stack, hence, in DeepESN architecture with L layers where information is processed hierachically have a matrix with *block lower-triangular* form, resulting in:

$$W_{\text{DeepESN}} = \begin{bmatrix} W_{\text{rec}}^{(1)} & \mathbf{0} & \cdots & \mathbf{0} \\ W_{\text{in}}^{(2)} & W_{\text{rec}}^{(2)} & \cdots & \mathbf{0} \\ \vdots & \ddots & \ddots & \vdots \\ \mathbf{0} & \cdots & W_{\text{in}}^{(L)} & W_{\text{rec}}^{(L)} \end{bmatrix} \quad (3.5)$$

Where each block on the diagonal represents the recurrent connections of each module, and the sub-diagonal blocks represent the input connections from the previous layer. When a single input perturbation is applied, deeper layers retain the perturbation for longer, indicating that higher layers capture slower (low-frequency) temporal components and lower layers capture faster (high-frequency) components, this is likely due to the fading memory effect of the reservoir dynamics. This indicates that increasing the distance between the input and subsequent layers is crucial for developing a rich spectrum of temporal dynamics

Qui ho chiamato an
che W_{in} le parti inte
layer dovrei cam-
biarne il nome per
non essere confuso
con la matrice che
manda l'input $u(t)$?

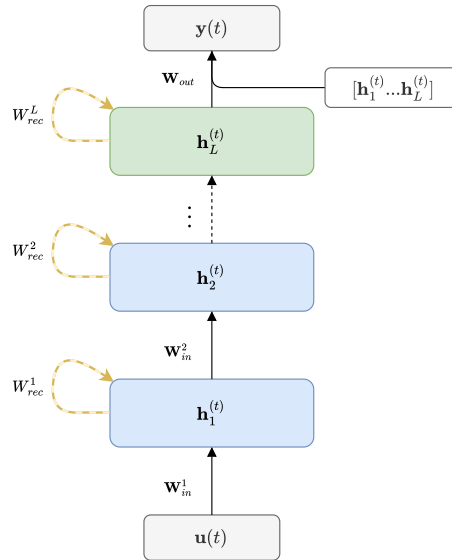


FIGURE 3.1: Modular structure of a standard DeepESN. Each layer acts as an independent module with feedforward inter-layer communication.

3.3 Deep Cycle Reservoir

We introduce **Deep Cycle Reservoir** (DCR) as class of deep reservoir, inspired by methods discussed in sec ?? . We tried to apply modularity concepts to a delayed version of SCR.

3.3.1 Motivation or Discussion?

In a conventional deepESN, information flows unidirectionally from the input through the stacked reservoirs to the readout. But it lacks a feedback path that might allow higher-level temporal abstractions to influence lower levels. A cyclic deep reservoir introduces a feedback connection from the output of the last reservoir layer back to the first layer. The motivation for closing the loop is to allow information processed at deeper layers (capturing longer-term or low-frequency features) to inform the processing of new inputs. This can enrich the reservoir dynamics and, in principle, improve tasks requiring both short and long-term memory.

However, a naïve instantaneous cycle where the output of the deepest layer at time step t is fed directly into the first layer at the same time step creates an algebraic loop. In the discrete-time update equation of an ESN, the next state depends on the previous state and the current input; if the current state of a high-level layer instantly modifies the state of the first layer, the system simultaneously depends on its own output. Such instantaneous feedback can destabilize the dynamics, (how...?).

Moreover, note that when each deepESN layer is fed the input directly (as in deepESN-IA [60]), the intrinsic time-scale separation is attenuated.

We can define the equation for the DCR as follows, omitting biases for simplicity:

$$\begin{aligned} \mathbf{h}^{(1)}(t) &= (1 - \alpha) \mathbf{h}^{(1)}(t-1) + \alpha \tanh \left(\mathbf{W}_{\text{in}}^{(1)} \mathbf{u}(t) + \mathbf{W}_{\text{rec}}^{(1)} \mathbf{h}^{(1)}(t-1) + \mathbf{W}_{\text{cyc}}^{(L)} \mathbf{h}^{(L)}(t-1) \right), \\ \mathbf{h}^{(l)}(t) &= (1 - \alpha) \mathbf{h}^{(l)}(t-1) + \alpha \tanh \left(\mathbf{W}_{\text{in}}^{(l)} \mathbf{u}(t) + \mathbf{W}_{\text{rec}}^{(l)} \mathbf{h}^{(l)}(t-1) + \mathbf{W}_{\text{cyc}}^{(l-1)} \mathbf{h}^{(l-1)}(t-1) \right). \end{aligned} \quad (3.6)$$

We have a global matrix W_{DCR} with:

$$W_{DCR} = \begin{bmatrix} \mathbf{W}_{\text{rec}}^{(1)} & \mathbf{0} & \dots & \mathbf{W}_{\text{cyc}}^{(L)} \\ \mathbf{W}_{\text{cyc}}^{(2)} & \mathbf{W}_{\text{rec}}^{(2)} & \dots & \mathbf{0} \\ \vdots & \ddots & \ddots & \vdots \\ \mathbf{0} & \dots & \mathbf{W}_{\text{cyc}}^{(L-1)} & \mathbf{W}_{\text{rec}}^{(L)} \end{bmatrix} \quad (3.7)$$

Where each $\mathbf{W}_{\text{rec}}^{(l)}$ is scaled by ρ and $\mathbf{W}_{\text{cyc}}^{(l)}$ is initialized as same as recurrent matrices but can have different ρ scaling factor. The ρ controls the effective spectral radius.²

3.3.2 Inter-layer coupling

We use a forward coupling between modules:

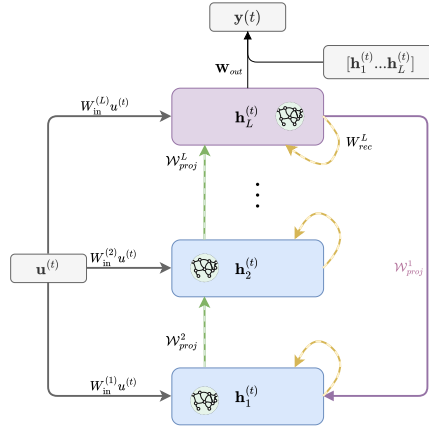


FIGURE 3.2: Deep Cycle Reservoir...

This creates a *delayed deep cycle reservoir* where each layer has a controlled internal topology and exchanges information with the previous layer in the cycle. As we can see from eq. ??, the first layer receives feedback from both the last layer at the previous timestep, thus closing the cycle, and the current input. For the remaining layers, each layer receives input at the current step and the previous layer's state at the previous step plus it's recurrent state.

Thus our architecture has a *ring topology* across depth, but is more similar to a DeepESN-IA [60] and a Delayed SCR [62]. We are losing the distance between input and deeper layer, but doing so, help to create a strong memory trace across depth.

3.3.3 Stability analysis of Deep Cycle Reservoir

In this section, we extend the analysis of standard ESN dynamics to the proposed DCR. We follow the theoretical framework established in [61], adapting the study of stability and contractivity to the specific case of cyclic modular connectivity. Is important to address the dynamics of such systems, since it affects both the capability to learn relevant information and keeping past information in the memory without losing it too quickly. We will study the stability of DCR, by linearizing its dynamics, and deriving conditions for ESP and asymptotic stability.

Finally we analyze the spectral radius of the system's Jacobian. Thus need to derive the Jacobian matrix $\mathbf{J}$ of equation ??.

²For a pure permutation cycle, the eigenvalues lie on the unit circle, and scaling by ρ sets their magnitude.

aggiungo immagini
di distanza tra stati
memory capacity?

forse questa parte v
messa dopo

Echo State Property of Deep Cycle Reservoir

To formally characterize the stability of the DCR, we adopt the definition of the ESP as formulated for deep architectures in [61]. We introduce the notation $\hat{F}$ to denote the iterated version of the DCR global state transition function defined in Eq. ??.

Given an input sequence of arbitrary finite length N_s , denoted as $\mathbf{s} \in (\mathbb{R}^{N_u})^*$, and a global initial state $\mathbf{H} \in \mathbb{R}^N$, the function $\hat{F}(\mathbf{s}, \mathbf{H})$ represents the global state of the network after being driven by the sequence $\mathbf{s}$ starting from $\mathbf{H}$.

The iterated map is defined recursively:

$$\hat{F} : (\mathbb{R}^{N_u})^* \times \mathbb{R}^N \rightarrow \mathbb{R}^N \quad (3.8)$$

$$\hat{F}(\mathbf{s}, \mathbf{H}) = \begin{cases} \mathbf{H} & \text{if } \mathbf{s} = [] \\ F(\mathbf{u}(k), \hat{F}([\mathbf{u}(1), \dots, \mathbf{u}(k-1)], \mathbf{H})) & \text{if } \mathbf{s} = [\mathbf{u}(1), \dots, \mathbf{u}(k)] \end{cases} \quad (3.9)$$

where $[]$ denotes the null sequence.

Accordingly, the ESP for the Deep Cycle Reservoir is defined as follows:

Definition 4 (Echo State Property of DCR). *Assuming a Deep Cycle Reservoir whose global dynamics are governed by the function F . The network has echo states if for any input sequence of length k , $\mathbf{s}_k = [\mathbf{u}(1), \dots, \mathbf{u}(k)]$, and for all pairs of initial global states $\mathbf{H}, \mathbf{H}' \in \mathbb{R}^N$, then:*

$$\|\hat{F}(\mathbf{s}_k, \mathbf{H}) - \hat{F}(\mathbf{s}_k, \mathbf{H}')\| \rightarrow 0 \quad \text{as } k \rightarrow \infty. \quad (3.10)$$

From a dynamical systems perspective, the DCR implements an input-driven discrete-time non-linear dynamical system F , mapping the external input $\mathbf{u}(t)$ and the global state $\mathbf{H}(t-1)$ to the next state $\mathbf{H}(t)$:

$$\mathbf{H}(t) = F(\mathbf{u}(t), \mathbf{H}(t-1)). \quad (3.11)$$

As defined in Eq. ??, the global state transition is governed by the global weight matrix $\mathbf{W}_{\text{cycle}}$ defined in ??.

Local Linearization of DCR Dynamics (Jacobian)

To investigate the stability of the DCR, we analyze the local first-order approximation of the system. The linearized dynamics of the DCR around a state $\mathbf{H}_0 \in \mathbb{R}^N$ (assuming zero input for stability analysis) are governed by the Jacobian matrix of the state transition function $\mathbf{J}(t) = \frac{\partial \mathbf{H}(t)}{\partial \mathbf{H}(t-1)}$.

Considering the block structure of the DCR, where connections exist not only between subsequent layers (feedforward) but also from the last layer to the first (feedback), the Jacobian $\mathbf{J}(t)$ takes the form of a **block-cyclic matrix**. Linearizing around the zero state ($\mathbf{H}_0 = \mathbf{0}$) where $\tanh'(0) = 1$, the Jacobian simplifies to (for a detailed derivation see Appendix ??):

$$\mathbf{J}(\mathbf{0}) = \begin{pmatrix} (1-\alpha)\mathbf{I} + \alpha\mathbf{W}_{\text{rec}}^{(1)} & \mathbf{0} & \cdots & \mathbf{0} & \alpha\mathbf{W}_{\text{cyc}}^{(1)} \\ \alpha\mathbf{W}_{\text{cyc}}^{(2)} & (1-\alpha)\mathbf{I} + \alpha\mathbf{W}_{\text{rec}}^{(2)} & \cdots & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \alpha\mathbf{W}_{\text{cyc}}^{(3)} & \ddots & \vdots & \vdots \\ \vdots & \ddots & \ddots & (1-\alpha)\mathbf{I} + \alpha\mathbf{W}_{\text{rec}}^{(L-1)} & \mathbf{0} \\ \mathbf{0} & \cdots & \mathbf{0} & \alpha\mathbf{W}_{\text{cyc}}^{(L)} & (1-\alpha)\mathbf{I} + \alpha\mathbf{W}_{\text{rec}}^{(L)} \end{pmatrix} \quad (3.12)$$

This can be written compactly as:

$$\mathbf{J}_{F,0} = (1-\alpha)\mathbf{I} + \alpha\mathbf{W}_{\text{DCR}}, \quad (3.13)$$

Theorem 1. (Necessary condition for the Deep Cycle Reservoir to satisfy the ESP of DCR). Assume a DCR as in ?? with global state transition function F . and a zero input. Then a necessary condition for ESP to hold is given by:

$$\rho(\mathbf{J}_{F,0}) = \rho((1-\alpha)\mathbf{I} + \alpha\mathbf{W}_{\text{DCR}}) < 1. \quad (3.14)$$

For a proof, see Appendix ??.

Remark on Coupling: unlike standard DeepESNs, where the spectrum is the union of the spectra of individual layers (due to the triangular Jacobian), the cycle in the DCR couples the eigenvalues of all layers. Consequently, $\rho(\mathbf{J}_{F,0})$ is **not** simply $\max_l \rho(\text{layer}_l)$. Stability in a DCR is a global property; an unstable feedback loop can drive the system to chaos even if individual modules satisfy the standard ESP condition locally.

Chapter 4

Implementation and Experimental results

Chapter 5

Experiments

This chapter describes the experimental protocol used to evaluate the proposed models. We consider (i) intrinsic dynamical properties (memory capacity) and (ii) supervised benchmarks spanning synthetic temporal regression (NARMA, Mackey–Glass) and sequential vision classification (MNIST, pMNIST, sMNIST, NP-CIFAR10).

5.1 General protocol

Datasets and splits. Unless specified otherwise, we use standard train/validation/test splits when available (e.g., MNIST, CIFAR-10). For synthetic sequences (NARMA, Mackey–Glass) we generate long sequences and carve out disjoint segments for training, validation, and testing to avoid temporal leakage.

Preprocessing. Inputs are normalized to zero mean and unit variance based on training statistics. For image-to-sequence tasks, pixels are mapped to a 1D stream as described in Sec. ??.

Model selection. Hyperparameters are selected on the validation set. We report the mean and standard deviation over multiple random seeds (initialization, data shuffling, and synthetic sequence generation).

Optimization. For trainable readouts, we use either (a) ridge regression (closed-form) for linear readouts or (b) gradient-based optimization (e.g., Adam) for end-to-end training. Early stopping is performed using validation performance. Exact hyperparameter grids and chosen values are reported in Sec. ??.

Metrics. For regression tasks we report normalized mean squared error (NMSE) and/or NRMSE. For classification tasks we report accuracy (%).

5.2 Intrinsic evaluation: Memory Capacity

We assess short-term memory via the linear *memory capacity* (MC) test. Let $u(t) \sim \mathcal{U}[-1, 1]$ be an i.i.d. input stream driving the system and let $x(t) \in \mathbb{R}^N$ be the state at time t .

For each delay $k \in \{1, \dots, K\}$ we train a linear readout $\hat{u}(t-k) = w_k^\top x(t)$ to reconstruct the delayed input. Define

$$\text{MC}_k = \frac{\text{cov}^2(u(t-k), \hat{u}(t-k))}{\text{var}(u(t-k)) \text{var}(\hat{u}(t-k))} \in [0, 1], \quad (5.1)$$

and the total memory capacity as

$$\text{MC} = \sum_{k=1}^K \text{MC}_k. \quad (5.2)$$

Training of readouts. We fit w_k using ridge regression with regularization λ selected on a validation segment. We discard an initial washout of T_{wash} steps to remove dependence on initial conditions.

Reported results. We report MC and optionally the curve MC_k vs. k to characterize how memory decays with delay.

5.3 Synthetic temporal regression tasks

5.3.1 NARMA- m

The Nonlinear AutoRegressive Moving Average task of order m is a standard benchmark for nonlinear memory. Given an input $u(t) \sim \mathcal{U}[0, 0.5]$, the target $y(t)$ is generated by

$$y(t+1) = 0.3y(t) + 0.05y(t) \left(\sum_{i=0}^{m-1} y(t-i) \right) + 1.5u(t-m+1)u(t) + 0.1. \quad (5.3)$$

We evaluate common orders (e.g., $m \in \{10, 20, 30\}$) and report NMSE:

$$\text{NMSE} = \frac{\frac{1}{T} \sum_t (y(t) - \hat{y}(t))^2}{\text{var}(y)}. \quad (5.4)$$

Protocol. We generate a long sequence, discard T_{wash} steps, train on T_{train} , validate on T_{val} , and report performance on a disjoint test segment of length T_{test} .

5.3.2 Mackey–Glass prediction

We consider one-step-ahead (or h -step-ahead) prediction of the Mackey–Glass (MG) time series, generated by the delay differential equation

$$\frac{dx(t)}{dt} = \beta \frac{x(t - \tau)}{1 + x(t - \tau)^n} - \gamma x(t), \quad (5.5)$$

with standard parameters (e.g., $\beta = 0.2$, $\gamma = 0.1$, $n = 10$, $\tau = 17$) and a numerical discretization step Δt . The supervised task is to predict $x(t + h)$ given the driven state at time t . We report NRMSE:

$$\text{NRMSE} = \sqrt{\frac{\frac{1}{T} \sum_t (x(t + h) - \hat{x}(t + h))^2}{\text{var}(x)}}. \quad (5.6)$$

5.4 Sequential vision classification

We cast image classification as a sequence modeling problem by feeding pixels (or small patches) sequentially.

5.4.1 MNIST

MNIST images are 28×28 grayscale. We consider:

- **sMNIST (sequential MNIST):** pixels are streamed row-wise as a length-784 sequence, $u(t) \in [0, 1]$.
- **pMNIST (permuted sequential MNIST):** a fixed random permutation π of $\{1, \dots, 784\}$ is applied to the pixel order, yielding $u(t) = \text{vec}(I)_{\pi(t)}$. The permutation is fixed across all samples and splits.

The label is provided at the end of the sequence; the model output is taken from the final state (or pooled states) and passed through a classifier.

Metric. We report test accuracy (%).

5.4.2 NP-CIFAR10 (sequential CIFAR-10 variant)

CIFAR-10 images are 32×32 with 3 channels. We define a sequential variant tailored to stress long-range dependencies:

- **Sequence construction:** flatten the image into a sequence of length $32 \cdot 32 = 1024$ tokens, where each token is a 3D vector (RGB) or a scalar after luminance projection.
- **Permutation (P):** apply a fixed random permutation of token indices to destroy locality.

Task	Type	Metric
Memory Capacity	intrinsic / linear readout	$MC = \sum_k MC_k$
NARMA- m	regression	NMSE / NRMSE
Mackey–Glass	regression	NRMSE
sMNIST	classification	accuracy (%)
pMNIST	classification	accuracy (%)
NP-CIFAR10	classification / robustness	accuracy (%), accuracy vs. noise

TABLE 5.1: Overview of experimental tasks and evaluation metrics.

- **Noise (N):** optionally add i.i.d. Gaussian noise $\epsilon \sim \mathcal{N}(0, \sigma^2)$ to inputs during training (and optionally testing) to assess robustness.

We refer to this setting as *NP-CIFAR10*. Exact choices (RGB vs. scalar, permutation seed, noise level σ) are specified in Sec. ??.

Metric. We report test accuracy (%). For noisy evaluations, we report accuracy vs. σ .

5.5 Implementation details and reproducibility

Hyperparameters. We document: state dimension N , input scaling, spectral radius / stability parameters (if applicable), leakage / time constants, regularization λ for ridge regression, learning rate, batch size, number of epochs, and early stopping patience.

Random seeds. All experiments are repeated over S seeds. We report mean $\pm$ standard deviation.

Compute. We report wall-clock training time, hardware (CPU/GPU model), and software versions (framework, CUDA, etc.) to support reproducibility.

5.6 Summary of reported results

Table ?? summarizes the tasks, outputs, and metrics.

Chapter 6

Conclusion and Future Works

Bibliography

- [1] Rajat Raina, Anand Madhavan, and Andrew Y. Ng. "Large-scale deep unsupervised learning using graphics processors". In: *Proceedings of the 26th Annual International Conference on Machine Learning*. ICML '09. Montreal, Quebec, Canada: Association for Computing Machinery, 2009, pp. 873–880. ISBN: 9781605585161. DOI: [10.1145/1553374.1553486](https://doi.org/10.1145/1553374.1553486). URL: <https://doi.org/10.1145/1553374.1553486>.
- [2] Jonathan Ho, Ajay Jain, and Pieter Abbeel. *Denoising Diffusion Probabilistic Models*. 2020. arXiv: [2006.11239](https://arxiv.org/abs/2006.11239) [cs.LG]. URL: <https://arxiv.org/abs/2006.11239>.
- [3] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. *Attention Is All You Need*. 2023. arXiv: [1706.03762](https://arxiv.org/abs/1706.03762) [cs.CL]. URL: <https://arxiv.org/abs/1706.03762>.
- [4] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Židek, Anna Potapenko, et al. "Highly accurate protein structure prediction with AlphaFold". In: *nature* 596.7873 (2021), pp. 583–589.
- [5] Sepp Hochreiter and Jürgen Schmidhuber. "Long Short-Term Memory". In: *Neural Computation* 9 (Nov. 1997), pp. 1735–1780. DOI: [10.1162/neco.1997.9.8.1735](https://doi.org/10.1162/neco.1997.9.8.1735).
- [6] Eric Renshaw. *Modelling biological populations in space and time*. 11. Cambridge University Press, 1993.
- [7] Rosario N. Mantegna and H. Eugene Stanley. *An introduction to econophysics: correlations and complexity in finance*. USA: Cambridge University Press, 1999. ISBN: 0521620082.
- [8] Mikhail I. Rabinovich, Pablo Varona, Allen I. Selverston, and Henry D. I. Abarbanel. "Dynamical principles in neuroscience". In: *Rev. Mod. Phys.* 78 (4 Nov. 2006), pp. 1213–1265. DOI: [10.1103/RevModPhys.78.1213](https://link.aps.org/doi/10.1103/RevModPhys.78.1213). URL: <https://link.aps.org/doi/10.1103/RevModPhys.78.1213>.
- [9] Rudolph Emil Kalman. "A new approach to linear filtering and prediction problems". In: (1960).
- [10] Leonard E Baum and Ted Petrie. "Statistical inference for probabilistic functions of finite state Markov chains". In: *The annals of mathematical statistics* 37.6 (1966), pp. 1554–1563.

- [11] Jeffrey L. Elman. "Finding structure in time". In: *Cognitive Science* 14.2 (1990), pp. 179–211. ISSN: 0364-0213. DOI: [https://doi.org/10.1016/0364-0213\(90\)90002-E](https://doi.org/10.1016/0364-0213(90)90002-E). URL: <https://www.sciencedirect.com/science/article/pii/S036402139090002E>.
- [12] E.M. Izhikevich. "Simple model of spiking neurons". In: *IEEE Transactions on Neural Networks* 14.6 (2003), pp. 1569–1572. DOI: [10.1109/TNN.2003.820440](https://doi.org/10.1109/TNN.2003.820440).
- [13] Steven H. Strogatz. *Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry and Engineering*. Westview Press, 2000.
- [14] Alan L Hodgkin and Andrew F Huxley. "A quantitative description of membrane current and its application to conduction and excitation in nerve". In: *The Journal of physiology* 117.4 (1952), p. 500.
- [15] L.R. Scott. *Numerical Analysis*. Princeton University Press, 2011. ISBN: 9781400838967. URL: https://books.google.it/books?id=SfCjL_5AaRQC.
- [16] Tom M Mitchell. *Machine learning*. Vol. 1. 9. McGraw-hill New York, 1997.
- [17] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. "Deep learning". In: *nature* 521.7553 (2015), pp. 436–444.
- [18] D. Barber. *Bayesian Reasoning and Machine Learning*. 04-2011. In press. Cambridge University Press, 2011. URL: <http://www.cs.ucl.ac.uk/staff/d.barber/brml>.
- [19] Vladimir N. Vapnik. *Statistical Learning Theory*. Wiley-Interscience, 1998.
- [20] Mehryar Mohri, Afshin Rostamizadeh, and Ameet Talwalkar. *Foundations of Machine Learning*. 2nd. The MIT Press, 2018. ISBN: 0262039400.
- [21] Matus Telgarsky. *Deep learning theory lecture notes*. <https://mjt.cs.illinois.edu/dlt/>. Version: 2021-10-27 v0.0-e7150f2d (alpha). 2021.
- [22] Francis Bach. *Learning theory from first principles*. MIT press, 2024.
- [23] Jorge Nocedal and Stephen J. Wright. *Numerical optimization*. 2. ed. Springer series in operations research and financial engineering. New York, NY: Springer, 2006. XXII, 664. ISBN: 978-0-387-30303-1. URL: http://gso.gbv.de/DB=2.1/CMD?ACT=SRCHA&SRT=YOP&IKT=1016&TRM=ppn+502988711&sourceid=fbw_bibsonomy.
- [24] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. <http://www.deeplearningbook.org>. MIT Press, 2016.
- [25] Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. 2017. arXiv: [1412.6980](https://arxiv.org/abs/1412.6980) [cs.LG]. URL: <https://arxiv.org/abs/1412.6980>.
- [26] D.Randall Wilson and Tony R. Martinez. "The general inefficiency of batch training for gradient descent learning". In: *Neural Networks* 16.10 (2003), pp. 1429–1451. ISSN: 0893-6080. DOI: [https://doi.org/10.1016/S0893-6080\(03\)00138-2](https://doi.org/10.1016/S0893-6080(03)00138-2). URL: <https://www.sciencedirect.com/science/article/pii/S0893608003001382>.
- [27] Vu Nguyen. "Bayesian Optimization for Accelerating Hyper-Parameter Tuning". In: *2019 IEEE Second International Conference on Artificial Intelligence and*

- Knowledge Engineering (AIKE)*. 2019, pp. 302–305. DOI: [10.1109/AIKE.2019.00060](https://doi.org/10.1109/AIKE.2019.00060).
- [28] Simon J.D. Prince. *Understanding Deep Learning*. MIT Press, 2023. URL: <http://udlbook.com>.
- [29] George Cybenko. “Approximation by superpositions of a sigmoidal function”. In: *Mathematics of control, signals and systems* 2.4 (1989), pp. 303–314.
- [30] Kunihiro Fukushima. “Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position”. In: *Biological cybernetics* 36.4 (1980), pp. 193–202.
- [31] Frank Rosenblatt. “The perceptron: a probabilistic model for information storage and organization in the brain.” In: *Psychological review* 65.6 (1958), p. 386.
- [32] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. “Learning representations by back-propagating errors”. In: *nature* 323.6088 (1986), pp. 533–536.
- [33] P. Tino, M. Cernansky, and L. Benuskova. “Markovian architectural bias of recurrent neural networks”. In: *IEEE Transactions on Neural Networks* 15.1 (2004), pp. 6–15. DOI: [10.1109/TNN.2003.820839](https://doi.org/10.1109/TNN.2003.820839).
- [34] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. *Sequence to Sequence Learning with Neural Networks*. 2014. arXiv: [1409.3215](https://arxiv.org/abs/1409.3215) [cs.CL]. URL: <https://arxiv.org/abs/1409.3215>.
- [35] Vijayaditya Peditinti, Daniel Povey, and Sanjeev Khudanpur. “A time delay neural network architecture for efficient modeling of long temporal contexts”. In: *Interspeech 2015*. 2015, pp. 3214–3218. DOI: [10.21437/Interspeech.2015-647](https://doi.org/10.21437/Interspeech.2015-647).
- [36] Paul J Werbos. “Backpropagation through time: what it does and how to do it”. In: *Proceedings of the IEEE* 78.10 (2002), pp. 1550–1560.
- [37] K. Doya. “Bifurcations in the learning of recurrent neural networks”. In: *[Proceedings] 1992 IEEE International Symposium on Circuits and Systems*. Vol. 6. 1992, 2777–2780 vol.6. DOI: [10.1109/ISCAS.1992.230622](https://doi.org/10.1109/ISCAS.1992.230622).
- [38] Y. Bengio, P. Simard, and P. Frasconi. “Learning long-term dependencies with gradient descent is difficult”. In: *IEEE Transactions on Neural Networks* 5.2 (1994), pp. 157–166. DOI: [10.1109/72.279181](https://doi.org/10.1109/72.279181).
- [39] Y. Bengio, P. Frasconi, and P. Simard. “The problem of learning long-term dependencies in recurrent networks”. In: *IEEE International Conference on Neural Networks*. 1993, 1183–1188 vol.3. DOI: [10.1109/ICNN.1993.298725](https://doi.org/10.1109/ICNN.1993.298725).
- [40] Simon S. Haykin. *Neural networks and learning machines*. Third. Upper Saddle River, NJ: Pearson Education, 2009.
- [41] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. “On the difficulty of training recurrent neural networks”. In: *Proceedings of the 30th International Conference on Machine Learning*. Ed. by Sanjoy Dasgupta and David McAllester. Vol. 28. Proceedings of Machine Learning Research 3. Atlanta, Georgia, USA:

- PMLR, 17–19 Jun 2013, pp. 1310–1318. URL: <https://proceedings.mlr.press/v28/pascanu13.html>.
- [42] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. *Deep Residual Learning for Image Recognition*. 2015. arXiv: 1512.03385 [cs.CV]. URL: <https://arxiv.org/abs/1512.03385>.
- [43] Martin Arjovsky, Amar Shah, and Yoshua Bengio. “Unitary Evolution Recurrent Neural Networks”. In: *Proceedings of The 33rd International Conference on Machine Learning*. Ed. by Maria Florina Balcan and Kilian Q. Weinberger. Vol. 48. Proceedings of Machine Learning Research. New York, New York, USA: PMLR, 20–22 Jun 2016, pp. 1120–1128. URL: <https://proceedings.mlr.press/v48/arjovsky16.html>.
- [44] Quoc V. Le, Navdeep Jaitly, and Geoffrey E. Hinton. *A Simple Way to Initialize Recurrent Networks of Rectified Linear Units*. 2015. arXiv: 1504.00941 [cs.NE]. URL: <https://arxiv.org/abs/1504.00941>.
- [45] Xavier Glorot, Antoine Bordes, and Yoshua Bengio. “Deep Sparse Rectifier Neural Networks”. In: *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*. Ed. by Geoffrey Gordon, David Dunson, and Miroslav Dudík. Vol. 15. Proceedings of Machine Learning Research. Fort Lauderdale, FL, USA: PMLR, Nov. 2011, pp. 315–323. URL: <https://proceedings.mlr.press/v15/glorot11a.html>.
- [46] Kyunghyun Cho, Bart Van Merriënboer, Dzmitry Bahdanau, and Yoshua Bengio. “On the properties of neural machine translation: Encoder-decoder approaches”. In: *arXiv preprint arXiv:1409.1259* (2014).
- [47] Razvan Pascanu, Caglar Gulcehre, Kyunghyun Cho, and Yoshua Bengio. *How to Construct Deep Recurrent Neural Networks*. 2014. arXiv: 1312.6026 [cs.NE]. URL: <https://arxiv.org/abs/1312.6026>.
- [48] Matthew D Zeiler and Rob Fergus. *Visualizing and Understanding Convolutional Networks*. 2013. arXiv: 1311.2901 [cs.CV]. URL: <https://arxiv.org/abs/1311.2901>.
- [49] Herbert Jaeger. “The “echo state” approach to analysing and training recurrent neural networks-with an erratum note”. In: *Bonn, Germany: German national research center for information technology gmd technical report 148.34* (2001), p. 13.
- [50] Chrisantha Fernando and Sampsa Sojakka. “Pattern recognition in a bucket”. In: *European conference on artificial life*. Springer. 2003, pp. 588–597.
- [51] Ronald J. Williams and David Zipser. “A Learning Algorithm for Continually Running Fully Recurrent Neural Networks”. In: *Neural Computation 1.2* (1989), pp. 270–280. DOI: 10.1162/neco.1989.1.2.270.
- [52] Thomas M Cover. “Geometrical and statistical properties of systems of linear inequalities with applications in pattern recognition”. In: *IEEE transactions on electronic computers 3* (2006), pp. 326–334.
- [53] Nils Bertschinger, Thomas Natschläger, and Robert Legenstein. “At the Edge of Chaos: Real-time Computations and Self-Organized Criticality in Recurrent

- Neural Networks". In: *Advances in Neural Information Processing Systems*. Ed. by L. Saul, Y. Weiss, and L. Bottou. Vol. 17. MIT Press, 2004. URL: https://proceedings.neurips.cc/paper_files/paper/2004/file/f8da71e562ff44a2bc7edf3578c593da-Paper.pdf.
- [54] Christopher G Langton. "Studying artificial life with cellular automata". In: *Physica D: Nonlinear Phenomena* 22.1 (1986). Proceedings of the Fifth Annual International Conference, pp. 120–149. ISSN: 0167-2789. DOI: [https://doi.org/10.1016/0167-2789\(86\)90237-X](https://doi.org/10.1016/0167-2789(86)90237-X). URL: <https://www.sciencedirect.com/science/article/pii/016727898690237X>.
- [55] Herbert Jaeger, Mantas Lukoševičius, Dan Popovici, and Udo Siewert. "Optimization and applications of echo state networks with leaky- integrator neurons". In: *Neural Networks* 20.3 (2007). Echo State Networks and Liquid State Machines, pp. 335–352. ISSN: 0893-6080. DOI: <https://doi.org/10.1016/j.neunet.2007.04.016>. URL: <https://www.sciencedirect.com/science/article/pii/S089360800700041X>.
- [56] Arthur E. Hoerl and Robert W. Kennard. "Ridge regression: biased estimation for nonorthogonal problems". In: *Technometrics* 42.1 (Feb. 2000), pp. 80–86. ISSN: 0040-1706. DOI: [10.2307/1271436](https://doi.org/10.2307/1271436). URL: <https://doi.org/10.2307/1271436>.
- [57] Roger A. Horn and Charles R. Johnson. *Matrix Analysis*. Cambridge University Press, 1990. ISBN: 0521386322. URL: <http://www.amazon.com/Matrix-Analysis-Roger-Horn/dp/0521386322%3FSubscriptionId%3D192BW6DQ43CK9FNOZGG2%26tag%3Dws%26linkCode%3Dxm2%26camp%3D2025%26creative%3D165953%26creativeASIN%3D0521386322>.
- [58] Matteo Cucci, Steven Abreu, Giuseppe Ciccone, Daniel Brunner, and Hans Kleemann. "Hands-on reservoir computing: a tutorial for practical implementation". In: *Neuromorphic Computing and Engineering* 2.3 (2022), p. 032002.
- [59] Herbert Jaeger. "Short Term Memory in Echo State Networks". In: (Jan. 2002).
- [60] Claudio Gallicchio, Alessio Micheli, and Luca Pedrelli. "Deep reservoir computing: A critical experimental analysis". In: *Neurocomputing* 268 (2017). Advances in artificial neural networks, machine learning and computational intelligence, pp. 87–99. ISSN: 0925-2312. DOI: <https://doi.org/10.1016/j.neucom.2016.12.089>. URL: <https://www.sciencedirect.com/science/article/pii/S0925231217307567>.
- [61] Claudio Gallicchio and Alessio Micheli. "Echo state property of deep reservoir computing networks". In: *Cognitive Computation* 9.3 (2017), pp. 337–350.
- [62] Ali Rodan and Peter Tino. "Minimum Complexity Echo State Network". In: *IEEE Transactions on Neural Networks* 22.1 (2011), pp. 131–144. DOI: [10.1109/TNN.2010.2089641](https://doi.org/10.1109/TNN.2010.2089641).
- [63] Bo Chang, Minmin Chen, Eldad Haber, and Ed H. Chi. *AntisymmetricRNN: A Dynamical System View on Recurrent Neural Networks*. 2019. arXiv: [1902.09689](https://arxiv.org/abs/1902.09689) [stat.ML]. URL: <https://arxiv.org/abs/1902.09689>.

- [64] Andrea Ceni, Andrea Cossu, Maximilian W Stölzle, Jingyue Liu, Cosimo Della Santina, Davide Bacciu, and Claudio Gallicchio. “Random Oscillators Network for Time Series Processing”. In: *Proceedings of The 27th International Conference on Artificial Intelligence and Statistics*. Ed. by Sanjoy Dasgupta, Stephan Mandt, and Yingzhen Li. Vol. 238. Proceedings of Machine Learning Research. PMLR, Feb. 2024, pp. 4807–4815. URL: <https://proceedings.mlr.press/v238/ceni24a.html>.
- [65] Ed Bullmore and Olaf Sporns. “Complex brain networks: graph theoretical analysis of structural and functional systems”. In: *Nature reviews neuroscience* 10.3 (2009), pp. 186–198.
- [66] Renaud Lambiotte and Michael T. Schaub. *Modularity and Dynamics on Complex Networks*. Elements in the Structure and Dynamics of Complex Networks. Cambridge University Press, 2022.

Appendices

Appendix A

Jacobian derivation of DCR

This appendix derives the Jacobian and both necessary and practical sufficient conditions for the Echo State Property (ESP) of the *Deep Cycle Reservoir* (DCR) described in Chapter ??.

A.1 Model definition

Let the input be $\mathbf{u}(t) \in \mathbb{R}^{N_u}$ and consider L reservoir modules with states

$$\mathbf{h}^{(l)}(t) \in \mathbb{R}^{n_l}, \quad l = 1, \dots, L.$$

The global reservoir state introduced in Eq. (??) is the concatenation

$$\mathbf{H}(t) := [\mathbf{h}^{(1)}(t), \dots, \mathbf{h}^{(L)}(t)] \in \mathbb{R}^N, \quad N = \sum_{l=1}^L n_l.$$

All modules share the same leak rate $\alpha \in (0, 1]$, and the element-wise nonlinearity is $\tanh(\cdot)$. The pre-activation of the first module matches Eq. (??):

$$\mathbf{z}^{(1)}(t) = \mathbf{W}_{\text{rec}}^{(1)} \mathbf{h}^{(1)}(t-1) + \mathbf{W}_{\text{in}}^{(1)} \mathbf{u}(t) + \mathbf{W}_{\text{cyc}}^{(L)} \mathbf{h}^{(L)}(t-1) + \mathbf{b}^{(1)}, \quad (\text{A.1})$$

$$\mathbf{z}^{(l)}(t) = \mathbf{W}_{\text{rec}}^{(l)} \mathbf{h}^{(l)}(t-1) + \mathbf{W}_{\text{in}}^{(l)} \mathbf{u}(t) + \mathbf{W}_{\text{cyc}}^{(l-1)} \mathbf{h}^{(l-1)}(t-1) + \mathbf{b}^{(l)}, \quad l = 2, \dots, L. \quad (\text{A.2})$$

Each module is updated through the leaky integration rule

$$\mathbf{h}^{(l)}(t) = (1 - \alpha) \mathbf{h}^{(l)}(t-1) + \alpha \tanh(\mathbf{z}^{(l)}(t)). \quad (\text{A.3})$$

Remark (time indexing for the inter-module links). All inter-module terms depend on $\mathbf{h}^{(\cdot)}(t-1)$, so the global map $\mathbf{H}(t) = F(\mathbf{u}(t), \mathbf{H}(t-1))$ is well-defined without introducing algebraic loops, exactly as discussed in Section ??.

A.2 Jacobian of the global state map of DCR

For a fixed input $\mathbf{u}(t)$, define the state transition map

$$\mathbf{H}(t) = F(\mathbf{u}(t), \mathbf{H}(t-1)).$$

The Jacobian with respect to the previous global state is

$$\mathbf{J}(t) := \frac{\partial \mathbf{H}(t)}{\partial \mathbf{H}(t-1)} \in \mathbb{R}^{N \times N}.$$

A.2.1 Layerwise derivatives

Let

$$\mathbf{D}^{(l)}(t) = \text{diag}(\tanh'(\mathbf{z}^{(l)}(t))) \in \mathbb{R}^{n_l \times n_l}.$$

From (??)–(??) we obtain, for the first module,

$$\frac{\partial \mathbf{h}^{(1)}(t)}{\partial \mathbf{h}^{(1)}(t-1)} = (1 - \alpha) \mathbf{I}_{n_1} + \alpha \mathbf{D}^{(1)}(t) \mathbf{W}_{\text{rec}}^{(1)}, \quad (\text{A.4})$$

$$\frac{\partial \mathbf{h}^{(1)}(t)}{\partial \mathbf{h}^{(L)}(t-1)} = \alpha \mathbf{D}^{(1)}(t) \mathbf{W}_{\text{cyc}}^{(L)}. \quad (\text{A.5})$$

For every deeper module $l \geq 2$ we have

$$\frac{\partial \mathbf{h}^{(l)}(t)}{\partial \mathbf{h}^{(l)}(t-1)} = (1 - \alpha) \mathbf{I}_{n_l} + \alpha \mathbf{D}^{(l)}(t) \mathbf{W}_{\text{rec}}^{(l)}, \quad (\text{A.6})$$

$$\frac{\partial \mathbf{h}^{(l)}(t)}{\partial \mathbf{h}^{(l-1)}(t-1)} = \alpha \mathbf{D}^{(l)}(t) \mathbf{W}_{\text{cyc}}^{(l-1)}. \quad (\text{A.7})$$

Introduce the block matrices

$$\mathbf{R}^{(l)}(t) := (1 - \alpha) \mathbf{I}_{n_l} + \alpha \mathbf{D}^{(l)}(t) \mathbf{W}_{\text{rec}}^{(l)}, \quad (\text{A.8})$$

$$\mathbf{P}^{(l)}(t) := \alpha \mathbf{D}^{(l)}(t) \mathbf{W}_{\text{cyc}}^{(l-1)}, \quad l = 2, \dots, L, \quad (\text{A.9})$$

$$\mathbf{C}^{(1)}(t) := \alpha \mathbf{D}^{(1)}(t) \mathbf{W}_{\text{cyc}}^{(L)}. \quad (\text{A.10})$$

A.2.2 Block structure

Let $\mathbf{J}_{l,k}(t) := \partial \mathbf{h}^{(l)}(t) / \partial \mathbf{h}^{(k)}(t-1)$. Using (??)–(??) we obtain

$$\mathbf{J}_{1,1}(t) = \mathbf{R}^{(1)}(t), \quad \mathbf{J}_{1,L}(t) = \mathbf{C}^{(1)}(t), \quad \mathbf{J}_{1,k}(t) = \mathbf{0} \quad (k \notin \{1, L\}), \quad (\text{A.11})$$

$$\mathbf{J}_{l,l}(t) = \mathbf{R}^{(l)}(t), \quad \mathbf{J}_{l,l-1}(t) = \mathbf{P}^{(l)}(t), \quad \mathbf{J}_{l,k}(t) = \mathbf{0} \quad (k \notin \{l, l-1\}), \quad (\text{A.12})$$

for $l = 2, \dots, L$. Consequently, the Jacobian has the block form

$$\mathbf{J}(t) = \begin{bmatrix} \mathbf{R}^{(1)}(t) & \mathbf{0} & \dots & \mathbf{0} & \mathbf{C}^{(1)}(t) \\ \mathbf{P}^{(2)}(t) & \mathbf{R}^{(2)}(t) & \ddots & \vdots & \mathbf{0} \\ \vdots & \ddots & \ddots & \mathbf{0} & \vdots \\ \mathbf{0} & \dots & \mathbf{P}^{(L)}(t) & \mathbf{R}^{(L)}(t) & \mathbf{0} \end{bmatrix},$$

which is block lower-bidiagonal with an additional non-zero block in the top-right corner coming from the cycle.

$$\mathbf{R}^{(l)} = (1 - \alpha)\mathbf{I}_{n_l} + \alpha\mathbf{W}_{\text{rec}}^{(l)}, \quad (\text{A.13})$$

$$\mathbf{P}^{(l)} = \alpha\mathbf{W}_{\text{cyc}}^{(l-1)}, \quad l = 2, \dots, L, \quad (\text{A.14})$$

$$\mathbf{C}^{(1)} = \alpha\mathbf{W}_{\text{cyc}}^{(L)}. \quad (\text{A.15})$$

For compactness, we define the global DCR weight matrix $\mathbf{W}_{\text{DCR}} \in \mathbb{R}^{N \times N}$ as

$$\mathbf{W}_{\text{DCR}} := \begin{bmatrix} \mathbf{W}_{\text{rec}}^{(1)} & \mathbf{0} & \dots & \mathbf{0} & \mathbf{W}_{\text{cyc}}^{(L)} \\ \mathbf{W}_{\text{cyc}}^{(1)} & \mathbf{W}_{\text{rec}}^{(2)} & \ddots & \vdots & \mathbf{0} \\ \vdots & \ddots & \ddots & \mathbf{0} & \vdots \\ \mathbf{0} & \dots & \mathbf{W}_{\text{cyc}}^{(L-1)} & \mathbf{W}_{\text{rec}}^{(L)} & \mathbf{0} \end{bmatrix}, \quad (\text{A.16})$$

where the off-diagonal blocks implement the cyclic couplings and the diagonal blocks are the intra-module recurrent weights.

Assuming a null input sequence $\mathbf{u}(t) = \mathbf{0}$ and analyzing the system around the zero state $\mathbf{H} = \mathbf{0}$, we have $\tanh(0) = 0$ and $\tanh'(0) = 1$, implying $\mathbf{D}^{(l)}(t) = \mathbf{I}_{n_l}$ at $\mathbf{H} = \mathbf{0}$. Substituting this into the block definitions (??)–(??) yields

With this notation, the Jacobian of the linearized system at $\mathbf{H} = \mathbf{0}$ can be written as

$$\mathbf{J}_{F,0} = (1 - \alpha)\mathbf{I} + \alpha\mathbf{W}_{\text{DCR}}. \quad (\text{A.17})$$

A.3 Special case: no cycle

If $\mathbf{W}_{\text{cyc}}^{(L)} = \mathbf{0}$ (and, more generally, $\mathbf{W}_{\text{cyc}}^{(l)} = \mathbf{0}$ for all l), the top-right block vanishes and $\mathbf{J}(t)$ becomes block lower-triangular. In this case the eigenvalues of $\mathbf{J}_{F,0}$ are simply the union of the spectra of the diagonal blocks. The ESP then reduces to the classical layer-wise requirement

$$\max_{l=1,\dots,L} \rho((1 - \alpha)\mathbf{I} + \alpha\mathbf{W}_{\text{rec}}^{(l)}) < 1,$$

recovered by dropping the coupling term in the sufficient condition (??) below.

A.4 Bounding the spectral radius via Gershgorin theorem

In practical RC scenario, often, only necessary conditions for the ESP can be verified, as sufficient conditions tend to be too restrictive. Here we provide a method to estimate an upper bound on the spectral radius of the DCR global weight matrix $\mathbf{W}_{\text{DCR}}$ defined in (??). One such bound is provided by the Gershgorin circle theorem.

Theorem 2 (Gershgorin Circle Theorem). *Let $\mathbf{A} \in \mathbb{C}^{N \times N}$ be a square matrix with entries a_{ij} . Every eigenvalue λ of $\mathbf{A}$ lies within at least one of the Gershgorin discs $D_i(\mathbf{A})$, defined as closed discs in the complex plane centered at a_{ii} with radius $R_i = \sum_{j \neq i} |a_{ij}|$. Consequently, the spectral radius is bounded by*

$$\rho(\mathbf{A}) \leq \max_i \left(|a_{ii}| + \sum_{j \neq i} |a_{ij}| \right). \quad (\text{A.18})$$

A.4.1 Application to the Deep Cycle Reservoir

We apply this theorem to the global weight matrix $\mathbf{W}_{\text{DCR}} \in \mathbb{R}^{N \times N}$ defined in (??). Although $\mathbf{W}_{\text{DCR}}$ has a block-cyclic structure, the Gershgorin theorem operates on each of the N scalar rows of the full matrix. Since $N = \sum_{l=1}^L n_l$, each row corresponds to a specific unit (i.e., a specific component of some module's state vector). Concretely, for the i -th unit within module l , the corresponding row of $\mathbf{W}_{\text{DCR}}$ contains:

- **Intra-module (recurrent) weights:** the i -th row of $\mathbf{W}_{\text{rec}}^{(l)}$, representing incoming connections *from* other units within the same module l to unit i ;
- **Inter-module (cycle) weights:** the i -th row of $\mathbf{W}_{\text{cyc}}^{\text{in}(l)}$, representing incoming connections *from* all units in the preceding module to unit i (where $\text{in}(l) = l - 1$ for $l > 1$, and $\text{in}(1) = L$).

For the i -th unit in module l , the diagonal entry of $\mathbf{W}_{\text{DCR}}$ is $(\mathbf{W}_{\text{rec}}^{(l)})_{ii}$, while the off-diagonal entries consist of all other recurrent weights in that row plus all cycle weights. Applying the Gershgorin bound yields

$$\rho(\mathbf{W}_{\text{DCR}}) \leq \max_{l=1, \dots, L} \left(\max_{i=1, \dots, n_l} \left\{ |(\mathbf{W}_{\text{rec}}^{(l)})_{ii}| + \sum_{k \neq i} |(\mathbf{W}_{\text{rec}}^{(l)})_{ik}| + \sum_{m=1}^{n_{\text{in}(l)}} |(\mathbf{W}_{\text{cyc}}^{\text{in}(l)})_{im}| \right\} \right), \quad (\text{A.19})$$

where $n_{\text{in}(l)}$ denotes the number of units in the module feeding into module l .